Code Explanation

GA Prompt Optimiser for LLM Code Generation

Evolving System Prompts on the APPS Dataset

Group 1

Computational Intelligence — Spring 2026

April 2026

Deep-dive walkthrough of every file, every function, and every call chain.

Includes exact function signatures, cross-file data-flow diagrams, and step-by-step worked examples tracing real data through the pipeline.

Designed for readers with no prior knowledge of genetic algorithms or LLM APIs.

Contents

1 Introduction **4**
- 1.1 What This Project Does . . . 4
- 1.2 The Codebase at a Glance . . . 4
- 1.3 Module Dependency Graph . . . 5
- 1.4 How to Read This Document . . . 5

2 Architecture & Data Flow **6**
- 2.1 The Two-Model Architecture . . . 6
- 2.2 Master Execution Pipeline . . . 6
- 2.3 Data Types Flowing Between Modules . . . 8
- 2.4 The API Call Budget . . . 8
- 2.5 Fitness Score: What Is Being Optimised? . . . 8

3 Configuration: `config.py` **10**
- 3.1 Purpose . . . 10
- 3.2 File at a Glance . . . 10
- 3.3 Full Source Code . . . 10
- 3.4 Block-by-Block Explanation . . . 12
  - 3.4.1 Path Resolution (lines 12–14) . . . 12
  - 3.4.2 Dataset Constants (lines 17–19) . . . 12
  - 3.4.3 Genetic Algorithm Constants (lines 22–28) . . . 12
  - 3.4.4 Mutation Probability Independence (lines 27–28) . . . 13
  - 3.4.5 LLM Constants (lines 31–43) . . . 13
  - 3.4.6 Execution Timeout (line 47) . . . 13

4 Seed Prompts: `seeds.py` **14**
- 4.1 Purpose . . . 14
- 4.2 File at a Glance . . . 14
- 4.3 Full Source Code . . . 14
- 4.4 Explanation . . . 17
  - 4.4.1 The 15 Strategies at a Glance . . . 17
  - 4.4.2 The Safety Assert . . . 17
  - 4.4.3 How Seeds Become Individuals . . . 17

5 Dataset Loading: `dataset.py` **19**
- 5.1 Purpose . . . 19
- 5.2 File at a Glance . . . 19
- 5.3 Full Source Code . . . 19
- 5.4 Function-by-Function Analysis . . . 21
  - 5.4.1 `load_problems(apps_train_dir)` . . . 21
  - 5.4.2 `split(problems, evolution_size, holdout_size, seed=42)` . . . 22

6 Code Evaluation: `evaluator.py` **24**
- 6.1 Purpose . . . 24
- 6.2 File at a Glance . . . 24
- 6.3 Full Source Code . . . 24

6.4 Function-by-Function Analysis . . . 27
6.4.1 extract_code(response) . . . 27
6.4.2 has_function(code, fn_name) . . . 28
6.4.3 _HARNESS_TEMPLATE . . . 28
6.4.4 _build_script(code, fn_name, inputs, outputs) . . . 29
6.4.5 run_tests(code, fn_name, inputs, outputs, timeout=5) . . . 30

7 Cost Tracking: cost_tracker.py **32**
7.1 Purpose . . . 32
7.2 File at a Glance . . . 32
7.3 Full Source Code . . . 32
7.4 Component-by-Component Analysis . . . 35
7.4.1 The PRICING Dictionary . . . 35
7.4.2 CallRecord Dataclass . . . 36
7.4.3 CostTracker Class . . . 36
7.4.4 The Singleton Pattern . . . 37

8 LLM Interface: llm.py **39**
8.1 Purpose . . . 39
8.2 File at a Glance . . . 39
8.3 Full Source Code . . . 39
8.4 Function-by-Function Analysis . . . 43
8.4.1 _get_client() . . . 43
8.4.2 _call(phase, model, temperature, max_tokens, system, user, retries=3) 44
8.4.3 generate_code(system_prompt, question, fn_name) . . . 45
8.4.4 crossover(prompt_a, prompt_b) . . . 46
8.4.5 mutate_inject(prompt) . . . 47
8.4.6 mutate_delete(prompt) . . . 47
8.4.7 mutate_rephrase(prompt) . . . 48

9 The Genetic Algorithm: ga.py **49**
9.1 Purpose . . . 49
9.2 File at a Glance . . . 49
9.3 Full Source Code . . . 49
9.4 Component-by-Component Analysis . . . 53
9.4.1 The Individual Dataclass . . . 53
9.4.2 compute_fitness(individual, problems) . . . 54
9.4.3 tournament_select(population, rng) . . . 55
9.4.4 breed(parent_a, parent_b, rng) . . . 55
9.4.5 run(seed_prompts, problems, on_generation=None) . . . 56

10 Entry Point: main.py **58**
10.1 Purpose . . . 58
10.2 File at a Glance . . . 58
10.3 Full Source Code . . . 58
10.4 Function-by-Function Analysis . . . 62
10.4.1 save_checkpoint(run_dir, gen, population, stats) . . . 62
10.4.2 evaluate_on_holdout(best_prompt, holdout) . . . 62
10.4.3 main() — The 7-Step Pipeline . . . 63
10.5 Output Files Reference . . . 64

11 Complete Worked Example: One Problem Through the Pipeline **65**
11.1 The Problem (on disk) 65
11.2 Step 1: `dataset.load_problems()` reads the problem 65
11.3 Step 2: `dataset.split()` assigns this problem to the evolution pool 66
11.4 Step 3: `ga.compute_fitness()` processes the problem 66
11.5 Step 4: `llm.generate_code()` sends the API request 66
11.6 Step 5: `llm._call()` makes the API request and logs cost 67
11.7 Step 6: `evaluator.extract_code()` strips the markdown 67
11.8 Step 7: `evaluator.run_tests()` executes and scores 68
11.9 Step 8: The score accumulates in `compute_fitness()` 69
11.10 Step 9: The result is recorded in the cost tracker 69

Glossary **71**

Chapter 1

Introduction

1.1 What This Project Does

This project uses a **Genetic Algorithm (GA)** to automatically discover the best "system prompt" for a Large Language Model (LLM) that generates Python code.

A **system prompt** is a paragraph of plain-English instructions prepended to every conversation with the LLM, shaping how it thinks before it sees the coding problem. Examples: *"Think like a competitive programmer"* or *"Always check edge cases before writing code."*

The GA treats each system prompt as a **chromosome** — a candidate solution that can be scored (fitness evaluated), combined with another (crossover), and lightly altered (mutation), exactly as in biological evolution. Over 20 generations the GA evolves a population of 15 prompts, keeping the best-performing ones and combining their strategies, until it converges on a prompt that makes the LLM solve the most coding problems correctly.

The Core Insight

Manual prompt engineering is trial-and-error. A human tries one prompt, tests it, tweaks it, and tries again. This project automates that loop: the GA *searches* the entire space of possible English instruction paragraphs by treating test-case pass rates as the fitness signal. The LLM's own creativity (via crossover and mutation calls) is used to *generate* new prompts, while its code-generation ability is what is being *optimised*.

1.2 The Codebase at a Glance

The project consists of 8 Python files totalling approximately 1,000 lines:

File	Lines	Public items	Role
`config.py`	62	16 constants	Single source of truth for all settings
`seeds.py`	116	1 list	The 15 generation-zero chromosomes
`dataset.py`	88	2 functions	Load & split APPS coding problems
`evaluator.py`	122	3 functions	Extract code; run tests in subprocess
`cost_tracker.py`	178	2 classes	Log token counts; compute API cost
`llm.py`	202	5 functions	All Claude API calls
`ga.py`	228	4 items	The full evolutionary loop
`main.py`	168	3 functions	Orchestrate, checkpoint, save results

Table 1.1: Project file summary.

1.3 Module Dependency Graph

Every `import` relationship is shown below. An arrow means "imports from".

Listing 1.1: Module import graph (arrows = imports).

```
main.py
  |-- imports --> config, dataset, evaluator, ga, llm
  |-- imports --> cost_tracker  (singleton: tracker)
  |-- imports --> seeds         (SEED_PROMPTS list)

ga.py
  |-- imports --> config, evaluator, llm

llm.py
  |-- imports --> config
  |-- imports --> cost_tracker  (singleton: tracker)
  |-- imports --> anthropic     (Anthropic API client)

evaluator.py
  |-- imports --> re, subprocess, sys  (stdlib only)

dataset.py
  |-- imports --> json, random, sys, pathlib  (stdlib only)

cost_tracker.py
  |-- imports --> json, time, dataclasses, datetime, pathlib  (stdlib only)

config.py
  |-- imports --> os, pathlib, dotenv

seeds.py
  |-- (no imports)
```

Key Import Insight

`evaluator.py`, `dataset.py`, `cost_tracker.py`, and `seeds.py` have *zero external dependencies* (only Python standard library or no imports at all). This means they can be tested in isolation without API keys or network access. `llm.py` is the *only* file that touches the Anthropic network.

1.4 How to Read This Document

Each chapter covers one file and follows this structure:

1. **Purpose** — plain-English summary of what the file does and why it exists.
2. **File at a Glance** — table of every function/class with one-line descriptions.
3. **Full Source Code** — the actual source with line numbers.
4. **Function-by-Function Analysis** — for every function:
   - Exact signature (all parameters, types, defaults)
   - What calls it and what it calls (the call chain)
   - Step-by-step walkthrough of the body
   - Return value with a concrete example
   - Edge cases and error handling
5. **Worked Example** — real data traced through the function.

Colour coding for boxes: **blue** = concept/theory; **orange** = warning/gotcha; **green** = function specification; **yellow** = worked example.

Chapter 2

Architecture & Data Flow

2.1 The Two-Model Architecture

A critical design choice is that *two separate Claude models* are used for two completely different purposes:

Role	Model	Temperature	Used for
Target (the subject)	`claude-haiku-4-5-20251001`	0.0 (deterministic)	Generating Python code (fitness
Optimizer (the tool)	`claude-sonnet-4-6`	0.7 (creative)	Crossover & mutation of prompt

Table 2.1: The two-model design.

Why Two Models?

Target (`Haiku 4.5`, temp=0): We are searching for the prompt that makes *this specific model* write the best code. Temperature 0 ensures that the same prompt always produces the same code on the same problem, making fitness scores *repeatable and comparable* across generations.

Optimizer (`Sonnet 4.6`, temp=0.7): Crossover and mutation require *creativity* — merging two prompts into a coherent paragraph, inventing a new instructional strategy. Sonnet is more capable for reasoning tasks; temperature 0.7 introduces enough randomness to explore new wording.

2.2 Master Execution Pipeline

Below is the complete call tree for a single run. Indentation shows nesting; line references are to the actual source files.

Listing 2.1: Complete call tree for one GA run (ga.py lines are approximate).

```
main.main()                                                [main.py:63]
  |
  |-- dataset.load_problems(config.APPS_TRAIN)             [dataset.py:18]
  |      Returns: list[dict]  (~2346 problems before filter)
  |
  |-- dataset.split(all_problems, 100, 200, seed=42)       [dataset.py:68]
  |      Returns: (evolution[100], holdout[200], rest[~2046])
  |
  |-- ga.run(SEED_PROMPTS, evolution_pool, on_generation=save_checkpoint)
  |                                                        [ga.py:146]
  |    |
  |    |-- [GEN 0: evaluate all 15 seeds]
  |    |    |
  |    |    +-- ga.compute_fitness(ind, problems)          [ga.py:50]  x15
  |    |         |
  |    |         +-- [for each of 100 problems]:
```

```
|    |              |
|    |              |-- llm.generate_code(            [llm.py:77]
|    |              |     system_prompt=ind.prompt,
|    |              |     question=prob["question"],
|    |              |     fn_name=prob["fn_name"])
|    |              |     |
|    |              |     +-- llm._call(              [llm.py:29]
|    |              |           phase="fitness",
|    |              |           model=TARGET_MODEL,
|    |              |           temperature=0.0,
|    |              |           max_tokens=1024,
|    |              |           system=ind.prompt,
|    |              |           user="<question>\n\nWrite a Python function
    named `fn_name`.")
|    |              |           |
|    |              |           |-- llm._get_client()  [llm.py:21]
|    |              |           |     Returns: anthropic.Anthropic instance (
    cached)
|    |              |           |
|    |              |           |-- client.messages.create(...)  [Anthropic
    API]
|    |              |           |     Returns: Message object
|    |              |           |
|    |              |           |-- tracker.record(    [cost_tracker.py:73]
|    |              |           |     phase, model,
|    |              |           |     input_tokens, output_tokens)
|    |              |           |     Appends CallRecord to self.calls
|    |              |           |
|    |              |           Returns: resp.content[0].text  (raw LLM string
    )
|    |              |
|    |              |-- evaluator.extract_code(response) [evaluator.py:24]
|    |              |     Returns: Python code string (stripped from markdown)
|    |              |
|    |              +-- evaluator.run_tests(           [evaluator.py:84]
|    |                    code, fn_name,
|    |                    inputs, outputs, timeout=5)
|    |                    |
|    |                    |-- evaluator.has_function(code, fn_name) [evaluator
    .py:43]
|    |                    |     Returns: bool
|    |                    |
|    |                    |-- evaluator._build_script( [evaluator.py:71]
|    |                    |     code, fn_name, inputs, outputs)
|    |                    |     Returns: complete Python script string
|    |                    |
|    |                    |-- subprocess.run([python, "-c", script], timeout
    =5)
|    |                    |     Returns: CompletedProcess (stdout with __R__:
    lines)
|    |                    |
|    |                    Returns: float in [0.0, 1.0]  (partial-credit score)
|    |
|    |-- [GENS 1..20: breed and evaluate]
|    |    |
|    |    +-- ga.tournament_select(population, rng)    [ga.py:84]  x2 per
    offspring
|    |    |     Returns: Individual (fittest of 3 random draws)
|    |    |
|    |    +-- ga.breed(parent_a, parent_b, rng)        [ga.py:94]  x13 per
    generation
|    |         |
```

```
|    |         |-- [80% chance] llm.crossover(         [llm.py:125]
|    |         |    parent_a.prompt, parent_b.prompt)
|    |         |    |
|    |         |    +-- llm._call(phase="crossover",
|    |         |          model=OPTIMIZER_MODEL, temp=0.7,
|    |         |          max_tokens=600, ...)
|    |         |    Returns: merged prompt string
|    |         |
|    |         |-- [25% chance] llm.mutate_inject(prompt)  [llm.py:147]
|    |         |    +-- llm._call(phase="mutate_inject", ...)
|    |         |    Returns: prompt with new strategy sentence added
|    |         |
|    |         |-- [15% chance] llm.mutate_delete(prompt)  [llm.py:166]
|    |         |    +-- llm._call(phase="mutate_delete", ...)
|    |         |    Returns: prompt with weakest sentence removed
|    |         |
|    |         |-- [10% chance] llm.mutate_rephrase(prompt) [llm.py:187]
|    |         |    +-- llm._call(phase="mutate_rephrase", ...)
|    |         |    Returns: same-meaning prompt in different words
|    |         |
|    |         Returns: Individual(prompt, parent_a.uid, parent_b.uid,
    operators)
|    |                  [fitness = -1.0, not yet evaluated]
|    |
|    |    +-- ga.compute_fitness(child, problems)  x13 [same sub-tree as above
    ]
|    |
|    |    +-- on_generation(gen-1, population, stats)
|    |         = main.save_checkpoint(run_dir, gen-1, population, stats)
|    |           Writes: results/run_TIMESTAMP/generation_NN.json
|    |
|    Returns: (best_Individual, history: list[dict])
|
|-- main.evaluate_on_holdout(best.prompt, holdout)     [main.py:48]
|     [Same sub-tree as compute_fitness but over 200 problems]
|     Returns: float (holdout fitness score)
|
+-- tracker.flush(extra={...})                          [cost_tracker.py:137]
      |-- CostTracker._by_phase()                       [cost_tracker.py:84]
      |-- CostTracker._by_model()                       [cost_tracker.py:109]
      Writes: Code/Costs/cost_TIMESTAMP.json
      Returns: Path object
```

2.3 Data Types Flowing Between Modules

2.4 The API Call Budget

To understand scale: one full run makes the following API calls:

2.5 Fitness Score: What Is Being Optimised?

The fitness of one prompt = the average partial-credit score across 100 APPS problems. The partial credit for one problem = (test cases passed) / (total test cases).

$$fitness(p) = \frac{1}{100}\sum_{i=1}^{100}\frac{testspassedonproblemiusingpromptp}{totaltestsonproblemi}$$

Producer	Consumer	Type	Description
`seeds.py`	`ga.py`	`list[str]`	15 initial prompt strings
`dataset.py`	`ga.py`, `main.py`	`list[dict]`	Problem dicts (id, question, fn_name, inputs, outputs)
`llm.py`	`ga.py`, `main.py`	`str`	Raw LLM response (may contain markdown)
`evaluator.py`	`ga.py`, `main.py`	`str`	Extracted Python code string
`evaluator.py`	`ga.py`, `main.py`	`float`	Partial-credit score in [0.0, 1.0]
`ga.py`	`main.py`	`Individual`	Best chromosome (prompt + fitness + metadata)
`ga.py`	`main.py`	`list[dict]`	History of best/mean/worst per generation
`llm.py`	`cost_tracker.py`	`int, int`	input_tokens, output_tokens per API call
`cost_tracker.py`	`main.py`	`Path`	Path to written cost JSON file

Table 2.2: Data flowing between modules.

Call type	Count	Model	Formula
Fitness (gen 0 seeds)	1,500	Haiku	15 seeds × 100 problems
Fitness (gens 1–20)	26,000	Haiku	20 gens × 13 offspring × 100 problems
Crossover	≤260	Sonnet	20 gens × 13 offspring × 80%
Mutate inject	≤325	Sonnet	20 gens × 13 offspring × 25%
Mutate delete	≤195	Sonnet	20 gens × 13 offspring × 15%
Mutate rephrase	≤130	Sonnet	20 gens × 13 offspring × 10%
Holdout evaluation	200	Haiku	1 × 200 problems
Total	**≈28,000**	—	—

Table 2.3: Approximate API call counts per full run.

A fitness of 1.0 means the prompt caused the LLM to write code that passed *every* test case on *every* problem. A fitness of 0.73 means 73% of test cases were passed on average.

Chapter 3

Configuration: `config.py`

3.1 Purpose

Every hyperparameter and file path lives in this one file. No other file hard-codes any numbers or paths. When you want to change the population size, reduce the number of generations, or switch Claude models, you edit *only* `config.py`.

3.2 File at a Glance

Name	Type	Description
`ROOT`	`Path`	Project root (one level above `Code/`)
`APPS_TRAIN`	`Path`	Absolute path to APPS training data
`RESULTS_DIR`	`Path`	Where run output folders are created
`EVOLUTION_SIZE`	`int`	Problems used for fitness during GA (100)
`HOLDOUT_SIZE`	`int`	Problems reserved for final test (200)
`RANDOM_SEED`	`int`	Seed for all random decisions (42)
`POPULATION_SIZE`	`int`	Chromosomes alive at any time (15)
`GENERATIONS`	`int`	Breeding cycles (20)
`ELITISM_COUNT`	`int`	Top individuals copied unchanged (2)
`TOURNAMENT_SIZE`	`int`	Contestants per parent selection (3)
`CROSSOVER_RATE`	`float`	Probability crossover fires per offspring (0.8)
`MUT_INJECT_PROB`	`float`	Probability inject mutation fires (0.25)
`MUT_DELETE_PROB`	`float`	Probability delete mutation fires (0.15)
`MUT_REPHRASE_PROB`	`float`	Probability rephrase mutation fires (0.10)
`ANTHROPIC_API_KEY`	`str`	API key read from `.env`
`TARGET_MODEL`	`str`	Claude model for code generation
`TARGET_TEMPERATURE`	`float`	Deterministic code generation (0.0)
`OPTIMIZER_MODEL`	`str`	Claude model for crossover/mutation
`OPTIMIZER_TEMPERATURE`	`float`	Creative prompt evolution (0.7)
`MAX_CODE_TOKENS`	`int`	Max tokens for code generation (1024)
`MAX_OPERATOR_TOKENS`	`int`	Max tokens for crossover/mutation (600)
`EXEC_TIMEOUT`	`int`	Seconds before subprocess is killed (5)

Table 3.1: `config.py`: all exported names.

3.3 Full Source Code

Listing 3.1: `config.py` — complete source.

```
"""
```

```python
config.py -- All hyperparameters and paths in one place.
Edit here; everything else reads from this module.
"""

import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()  # reads .env file in the current working directory

# Paths
ROOT          = Path(__file__).parent.parent
APPS_TRAIN    = ROOT / "Dataset Examples" / "APPS" / "train"
RESULTS_DIR   = Path(__file__).parent / "results"

# Dataset
EVOLUTION_SIZE = 100   # fixed pool for fitness evaluation during GA
HOLDOUT_SIZE   = 200   # held-out for final within-distribution test
RANDOM_SEED    = 42

# Genetic Algorithm
POPULATION_SIZE   = 15
GENERATIONS       = 20
ELITISM_COUNT     = 2   # top-N individuals carried unchanged to next generation
TOURNAMENT_SIZE   = 3   # contestants per tournament selection draw

CROSSOVER_RATE      = 0.8   # probability crossover is applied to each offspring

# Independent per-individual mutation probabilities (can all apply at once)
MUT_INJECT_PROB   = 0.25   # add a new instructional strategy
MUT_DELETE_PROB   = 0.15   # drop a random sentence
MUT_REPHRASE_PROB = 0.10   # rewrite without changing meaning

# LLM
ANTHROPIC_API_KEY = os.environ.get("ANTHROPIC_API_KEY", "")

# Model being optimised -- generates code, evaluated for fitness.
TARGET_MODEL       = "claude-haiku-4-5-20251001"
TARGET_TEMPERATURE = 0.0   # deterministic for repeatable fitness scores

# Model used for crossover / mutation meta-prompts -- needs creativity.
OPTIMIZER_MODEL       = "claude-sonnet-4-6"
OPTIMIZER_TEMPERATURE = 0.7

MAX_CODE_TOKENS     = 1024
MAX_OPERATOR_TOKENS = 600

# Code execution
EXEC_TIMEOUT = 5   # seconds per subprocess evaluation
```

3.4 Block-by-Block Explanation

3.4.1 Path Resolution (lines 12–14)

Path constants

Signature: Module-level constants, resolved at import time.
Called by: `dataset.load_problems()` reads `APPS_TRAIN`; `main.main()` reads `RESULTS_DIR`; `main.main()` checks `APPS_TRAIN.exists()`.

```
ROOT        = Path(__file__).parent.parent
               ^-- __file__ = "/home/.../Code/config.py"
               ^-- .parent  = "/home/.../Code/"
               ^-- .parent  = "/home/.../Project/"

APPS_TRAIN  = ROOT / "Dataset Examples" / "APPS" / "train"
            = "/home/.../Project/Dataset Examples/APPS/train"

RESULTS_DIR = Path(__file__).parent / "results"
            = "/home/.../Code/results"
```

`Path(__file__)` gives the absolute path of `config.py` itself. Chaining `.parent` moves up one directory level. The `/` operator on `Path` objects joins segments — it works on all operating systems without needing to worry about forward vs. backward slashes.

3.4.2 Dataset Constants (lines 17–19)

- `EVOLUTION_SIZE = 100` — The GA evaluates every prompt on exactly these 100 problems. Keeping the set fixed across all generations is critical: if the problems changed, a prompt might score higher simply because it got easier problems, not because it improved.
- `HOLDOUT_SIZE = 200` — After the GA finishes, the *best* prompt is tested on these 200 *never-seen* problems to measure generalisation. This plays the same role as a test set in machine learning.
- `RANDOM_SEED = 42` — Passed to `dataset.split()` and `ga.run()`'s `random.Random` instance to make the experiment fully reproducible.

3.4.3 Genetic Algorithm Constants (lines 22–28)

Elitism prevents regression

Without elitism, a great prompt discovered in generation 5 might be replaced by worse offspring in generation 6. `ELITISM_COUNT = 2` guarantees the top 2 prompts always survive into the next generation *unchanged*, so the population's best fitness can only stay the same or improve.

Tournament size: exploration vs. exploitation

`TOURNAMENT_SIZE = 3` means we draw 3 random contestants and select the best. A smaller tournament (e.g. 2) gives weaker individuals more chances, increasing exploration. A larger tournament (e.g. 10) almost always picks the global best, reducing exploration. Size 3 is a standard balanced choice.

3.4.4 Mutation Probability Independence (lines 27–28)

All three mutation probabilities are *independent*: each is checked separately with its own `rng.random()` call. On any single offspring, any combination of all three (or none) can fire, giving $2^3 = 8$ possible operator combinations.

Combination	Probability	Resulting operators string
none	$(1-0.25)(1-0.15)(1-0.10) = 0.574$	`"crossover"` or `"copy"`
inject only	$0.25 \times 0.85 \times 0.90 = 0.191$	`"+inject"`
delete only	$0.75 \times 0.15 \times 0.90 = 0.101$	`"+delete"`
rephrase only	$0.75 \times 0.85 \times 0.10 = 0.064$	`"+rephrase"`
inject+delete	$0.25 \times 0.15 \times 0.90 = 0.034$	`"+inject+delete"`
inject+rephrase	...	`"+inject+rephrase"`
delete+rephrase	...	`"+delete+rephrase"`
all three	$0.25 \times 0.15 \times 0.10 = 0.00375$	`"+inject+delete+rephrase"`

Table 3.2: Possible mutation combinations (probabilities are for crossover only; if copy fires, similar logic applies).

3.4.5 LLM Constants (lines 31–43)

`ANTHROPIC_API_KEY` is read from the environment (or from a `.env` file via `load_dotenv()`) — never hard-coded. If the key is absent, `main.main()` catches this and exits with a clear error message before wasting time loading the dataset.

`MAX_CODE_TOKENS = 1024`: the target LLM's response for a code problem is limited to 1,024 tokens (roughly 750 words of code). If the LLM tries to write a longer response, the API truncates it — which would produce syntactically broken code that fails all tests.

`MAX_OPERATOR_TOKENS = 600`: crossover and mutation prompts produce shorter text (a modified instruction paragraph), so 600 tokens is sufficient and cheaper.

3.4.6 Execution Timeout (line 47)

`EXEC_TIMEOUT = 5`: each test run gets 5 seconds. If the LLM-generated code contains an infinite loop (a common bug), the subprocess is forcibly killed after 5 seconds and the problem scores 0.0. Without this, one bad code snippet could halt the entire GA.

Chapter 4

Seed Prompts: `seeds.py`

4.1 Purpose

The GA must start with a population. `seeds.py` provides 15 hand-crafted instruction prompts — the "generation zero" chromosomes. Each emphasises a different cognitive strategy for solving coding problems so the GA starts with genuinely diverse material.

Why diversity in seeds matters

If all 15 seeds said the same thing (e.g., "Write correct Python code"), crossover would merely re-shuffle identical text and mutation would be the *only* source of novelty. By seeding 15 different strategies — edge-case thinking, algorithm identification, pseudocode-first, output-format checking, etc. — crossover can combine genuinely different approaches. A hybrid of "analyse edge cases first" and "identify the algorithm pattern" can be better than either parent alone.

4.2 File at a Glance

Name	Type	Description
`SEED_PROMPTS`	`list[str]`	15 diverse instruction strings (gen-0 chromosomes)

Table 4.1: `seeds.py`: exported names.

4.3 Full Source Code

Listing 4.1: `seeds.py` — all 15 seed prompts.

```python
"""
seeds.py -- Initial population: 15 diverse system-level instruction prompts.

These are the chromosomes fed to generation 0.  Diversity is intentional --
each emphasises a different coding strategy so the GA has a rich search
starting point.  The GA will crossover and mutate these into better prompts.
"""

SEED_PROMPTS: list[str] = [
    # 1 -- Structured analysis
    (
```

```
        "You are an expert Python programmer. Before writing code, analyse the "
        "problem: identify the input types, the expected output type, and any "
        "edge cases (empty input, zero, negatives). Then write a clean, correct "
        "Python function. Output only the function -- no explanations. "
    ),

    # 2 -- Competitive-programmer mindset
    (
        "Think like a competitive programmer. Identify the algorithm pattern "
        "(math, string manipulation, recursion, sorting, hashing). Write the "
        "most direct, correct Python solution. Output only the function definition."
    ),

    # 3 -- Return-type focus
    (
        "You are solving a coding kata. Read the problem twice. Pay close "
        "attention to what the function must return -- the exact type and format. "
        "Handle edge cases. Output just the function."
    ),

    # 4 -- Step-by-step reasoning
    (
        "Approach this problem step by step: (1) understand the input-output "
        "relationship, (2) identify the core algorithm, (3) handle boundary "
        "conditions, (4) implement cleanly in Python. Return only the function."
    ),

    # 5 -- Example-driven
    (
        "Before coding, trace through the provided examples manually to verify "
        "your understanding of the expected behaviour. Then implement a clean "
        "Python function that passes all cases."
    ),

    # 6 -- Minimal and precise
    (
        "Write a correct Python function. Avoid unnecessary imports and extra "
        "code. The simplest solution that handles all cases is the best solution."
    ),

    # 7 -- Edge-case defender
```

```
    (
        "You are a careful Python programmer. Explicitly think about boundary "
        "conditions: empty input, single element, zero, negative values, and "
        "the maximum possible input. Write a function that handles all of them."
    ),

    # 8 -- Algorithm-first
    (
        "Determine the most appropriate algorithm or data structure before "
        "writing any code. Implement it directly in Python. Avoid "
        "over-engineering -- correctness and clarity are the priorities."
    ),

    # 9 -- Output-format guard
    (
        "Solve this Python coding problem precisely. Check the expected return "
        "type carefully -- integer, list, string, boolean, or tuple. Your "
        "function must return exactly that type. Output only the function."
    ),

    # 10 -- Pseudocode-then-code
    (
        "First mentally sketch the algorithm in plain English, then translate "
        "it directly into Python. Keep the implementation close to the "
        "pseudocode so it is easy to verify. Output only the function. "
    ),

    # 11 -- Constraint-aware
    (
        "Read the problem constraints carefully. Choose an algorithm whose "
        "time complexity fits within those constraints. Implement it cleanly "
        "in Python and return only the function definition."
    ),

    # 12 -- Defensive / robust
    (
        "Write a robust Python function. Assume the input can be anything "
        "within the stated constraints -- do not assume best-case input. "
        "Handle all valid inputs correctly and return the right value. "
    ),

```

```
94      # 13 -- Readability-first
95      (
96          "Write clean, readable Python code. Use meaningful variable
               names and "
97          "keep the logic straightforward. Correct and readable code
               beats "
98          "clever but obscure code. Output only the function."
99      ),
100
101     # 14 -- Verify-against-examples
102     (
103         "Expert Python programmer: analyse the input constraints, pick
                the "
104         "right algorithm, and mentally verify your solution against
               the given "
105         "examples before finalising. Output only the function."
106     ),
107
108     # 15 -- Direct and focused
109     (
110         "Write a Python function to solve the given problem. Think
               about "
111         "what transformation is needed, handle edge cases, and
               implement the "
112         "simplest correct solution. No test code, no extra output."
113     ),
114 ]
115
116 assert len(SEED_PROMPTS) == 15, "Need exactly 15 seed prompts (=
       POPULATION_SIZE)"
```

4.4 Explanation

4.4.1 The 15 Strategies at a Glance

4.4.2 The Safety Assert

Listing 4.2: The guard at the bottom of seeds.py (line 116).

```
assert len(SEED_PROMPTS) == 15, "Need exactly 15 seed prompts (= POPULATION_SIZE
    )"
```

If you add or remove a seed, Python immediately raises `AssertionError` when `seeds.py` is imported — before any API calls are made or any money is spent. The error message reminds you to match `POPULATION_SIZE` in `config.py`.

4.4.3 How Seeds Become Individuals

In `ga.run()` [ga.py:167]:

```
population = [Individual(prompt=p) for p in seed_prompts]
# Each seed string becomes an Individual with:
#   prompt   = the seed string (e.g. "Think like a competitive programmer...")
#   fitness  = -1.0  (unevaluated sentinel)
#   uid      = random 8-char hex (e.g. "a3f7c2b1")
#   parent_a = ""  (empty: seeds have no parents)
```

#	Strategy	Core instruction focus
1	Structured analysis	Identify input types, output type, edge cases *before* coding
2	Competitive programmer	Find the algorithm pattern; write direct correct solution
3	Return-type focus	Read twice; match exact return type and format
4	Step-by-step reasoning	4-step process: understand, identify, handle, implement
5	Example-driven	Trace examples manually to verify understanding
6	Minimal and precise	Simplest solution; avoid unnecessary code
7	Edge-case defender	Explicitly enumerate: empty, single, zero, negative, max
8	Algorithm-first	Choose data structure *before* writing a line
9	Output-format guard	Check return type (int/list/str/bool/tuple) explicitly
10	Pseudocode-then-code	English sketch first, then translate to Python
11	Constraint-aware	Match time complexity to stated constraints
12	Defensive / robust	Assume worst-case valid input; never assume best-case
13	Readability-first	Meaningful names; correct & readable beats clever
14	Verify-against-examples	Mental verification against examples before finalising
15	Direct and focused	Transformation-first; no test code; no extra output

Table 4.2: The 15 seed strategies.

```
#   parent_b = ""
#   operators = ""
```

Chapter 5

Dataset Loading: `dataset.py`

5.1 Purpose

This file loads coding problems from the APPS dataset (stored on disk as folders of JSON files) and splits them into three non-overlapping groups: the **evolution pool** (100 problems used as the fitness benchmark during the GA), the **held-out set** (200 problems reserved for the final generalisation test), and the **rest** (unused buffer).

5.2 File at a Glance

Function	Returns	Description
`load_problems(apps_train_dir)`	`list[dict]`	Reads all APPS pro Codewars introduct
`split(problems, evolution_size, holdout_size, seed=42)`	`(list, list, list)`	Deterministic shuff / holdout / rest

Table 5.1: `dataset.py`: public functions.

5.3 Full Source Code

Listing 5.1: `dataset.py` — complete source.

```python
"""
dataset.py -- Load and split APPS Codewars introductory fn_name problems.

Why Codewars only?
    * 2,346 introductory problems from codewars.com, all standalone functions.
    * Consistent execution format:
        inputs[i]  = Python list of arguments already parsed (e.g. [1] or [2, "x"])
        outputs[i] = [return_value]  -- the result wrapped in a single-element list
    * LeetCode-style problems use class Solution, which complicates extraction.
"""

import json
import random
import sys
from pathlib import Path

```

```python

def load_problems(apps_train_dir: str | Path) -> list[dict]:
    """
    Return every Codewars introductory fn_name problem from the APPS train set.

    Each problem dict has:
        id         - folder name (e.g. "3715")
        question   - full problem text
        fn_name    - function the LLM must write
        inputs     - list of test-case argument lists
        outputs    - list of [expected_return_value]
    """
    sys.set_int_max_str_digits(1_000_000)   # needed for a handful of problems
    problems = []
    train_dir = Path(apps_train_dir)

    for prob_dir in train_dir.iterdir():
        meta_f = prob_dir / "metadata.json"
        io_f   = prob_dir / "input_output.json"
        q_f    = prob_dir / "question.txt"

        if not (meta_f.exists() and io_f.exists() and q_f.exists()):
            continue

        try:
            meta = json.loads(meta_f.read_text())
            io   = json.loads(io_f.read_text())
        except Exception:
            continue   # skip malformed JSON

        # Keep only: introductory difficulty, fn_name style, Codewars source
        if (
            meta.get("difficulty") != "introductory"
            or "fn_name" not in io
            or "codewars.com" not in meta.get("url", "")
            or not io.get("inputs")
            or not io.get("outputs")
        ):
            continue

        problems.append({
            "id":       prob_dir.name,
            "question": q_f.read_text().strip(),
            "fn_name":  io["fn_name"],
            "inputs":   io["inputs"],
            "outputs":  io["outputs"],
        })

    return problems


def split(
    problems: list[dict],
    evolution_size: int,
```

```
    holdout_size: int,
    seed: int = 42,
) -> tuple[list[dict], list[dict], list[dict]]:
    """
    Shuffle and split into:
        evolution -- used for fitness evaluation every generation
        holdout   -- unseen until final evaluation
        rest      -- remaining problems (not used during training)
    """
    rng = random.Random(seed)
    shuffled = problems.copy()
    rng.shuffle(shuffled)

    evolution = shuffled[:evolution_size]
    holdout   = shuffled[evolution_size : evolution_size + holdout_size]
    rest      = shuffled[evolution_size + holdout_size :]

    return evolution, holdout, rest
```

5.4 Function-by-Function Analysis

5.4.1 load_problems(apps_train_dir)

load_problems(apps_train_dir: str | Path) -> list[dict]

Defined: dataset.py, line 18
Called by: main.main() [main.py:75]
Calls: Path.iterdir(), json.loads(), Path.read_text(), sys.set_int_max_str_digits()
Parameters:
apps_train_dir — string or Path to the APPS train/ directory
Returns: list[dict] — one dict per qualifying problem (typically ≈2,346 items)

Step-by-step walkthrough:

1. **Line 29**: sys.set_int_max_str_digits(1_000_000) — some APPS problems have very large integer outputs. Python 3.11+ limits integer-to-string conversions for security; this raises the limit so those problems don't crash.

2. **Line 33**: train_dir.iterdir() — yields each subdirectory (each problem folder, e.g. 3715/, 4820/, etc.).

3. **Lines 34–36**: Construct paths to the 3 files inside each problem folder:
   - metadata.json — contains difficulty, url, source
   - input_output.json — contains fn_name, inputs, outputs
   - question.txt — the problem statement in plain text

4. **Lines 38–39**: **Existence check** — skip any folder missing one of the three files. Some APPS folders are incomplete.

5. **Lines 41–45**: **JSON parsing** inside a try/except — skips folders with malformed or empty JSON without crashing.

6. **Lines 47–55**: **Five-condition filter** (all must be true):
   (a) `difficulty == "introductory"` — exclude interview/competition problems
   (b) `"fn_name" in io` — exclude stdin/stdout style problems
   (c) `"codewars.com" in url` — exclude LeetCode, Codeforces, etc.
   (d) `io["inputs"]` is non-empty — must have test inputs
   (e) `io["outputs"]` is non-empty — must have expected outputs
7. **Lines 57–63**: Build the **problem dict** and append to the list.

Worked Example — A real problem dict

```
{
    "id":       "3715",
    "question": "Complete the function that takes an array of words.\n"
                "You must concatenate the nth letter from each word using "
                "the enumerating order of the word...",
    "fn_name":  "nth_char",
    "inputs":   [
        [["yoda", "best", "has"]],    # test case 1: one argument (a list)
        [["a", "bc", "def"]]          # test case 2
    ],
    "outputs":  [
        ["oes"],                       # expected return for test case 1
        ["abd"]                        # expected return for test case 2
    ]
}
# Note: inputs[i] is a list of arguments; fn_name(*inputs[i]) unpacks them.
# Note: outputs[i] is [return_value] (always wrapped in a single-element list
    ).
```

Why Codewars only? The APPS dataset mixes problems from many sources. Codewars is chosen for three reasons:

1. **Standalone function format** — every problem defines `def fn_name(...): return ...`. LeetCode uses `class Solution` methods, which require different extraction logic.
2. **Consistent I/O format** — inputs are already-parsed Python lists of arguments; outputs are `[return_value]`. This exact format is what the evaluator's harness expects.
3. **Quantity** — ≈2,346 qualifying problems, far more than the 100 + 200 needed.

5.4.2 `split(problems, evolution_size, holdout_size, seed=42)`

`split(problems, evolution_size, holdout_size, seed=42) -> (list, list, list)`

Defined: `dataset.py`, line 68
Called by: `main.main()` [main.py:82]
Calls: `random.Random()`, `list.copy()`, `random.Random.shuffle()`
Parameters:
problems — full list from `load_problems()` (≈2,346 dicts)
evolution_size — size of the evolution pool (100)
holdout_size — size of the held-out test set (200)
seed — RNG seed for reproducible split (42)
Returns: `tuple[list, list, list]` — (evolution, holdout, rest)

Step-by-step walkthrough:

1. **Line 80**: `random.Random(seed)` creates an *isolated* RNG with seed 42. This is separate from Python's global `random` module, so other calls to `random` elsewhere don't affect the shuffle.

2. **Line 81**: `problems.copy()` — makes a shallow copy so the original list order is preserved (the caller may want it unchanged).

3. **Line 82**: `rng.shuffle(shuffled)` — shuffles in place, deterministically (same seed always produces same order).

4. **Lines 84–86**: Slice into three non-overlapping segments:
   - `evolution = shuffled[:100]` — problems 0–99
   - `holdout = shuffled[100:300]` — problems 100–299
   - `rest = shuffled[300:]` — problems 300–∼2,346 (unused)

Worked Example — Determinism is critical

```
# Every single run, this call produces the SAME 100 evolution problems:
evolution, holdout, rest = dataset.split(
    all_problems, 100, 200, seed=42
)

# If seed=42 always gives the same shuffle, then:
# - Fitness scores in generation 3 are comparable to generation 7.
# - The held-out set is genuinely unseen (never used for selection).
# - Two researchers can reproduce the exact same experiment.

# If we used random.shuffle() (global RNG, not seeded), fitness scores
# from different generations would not be comparable because the
# 100 problems would change every run.
```

Train/Test Split — same idea as machine learning

In ML, you train a model on training data and evaluate it on held-out test data to measure generalisation. Here, the GA "trains" on the 100-problem evolution pool (the fitness signal) and the best evolved prompt is evaluated on the 200-problem holdout to confirm it generalises to unseen problems rather than overfitting to the specific 100.

Chapter 6

Code Evaluation: `evaluator.py`

6.1 Purpose

When the LLM generates code in response to a prompt, we need to:

1. **Extract** the Python code from the LLM's raw text response (which may contain markdown fences, explanations, and other text).
2. **Execute** the extracted code against test cases in a *completely isolated subprocess* and return how many tests passed.

This file has no LLM calls and no imports beyond the Python standard library.

6.2 File at a Glance

Item	Type	Description
`extract_code(response)`	function	Pull Python code from raw LLM text
`has_function(code, fn_name)`	function	Check if the required function is defin
`_HARNESS_TEMPLATE`	str	Template for the subprocess test scrip
`_build_script(code, fn_name, inputs, outputs)`	function	Assemble the complete test script
`run_tests(code, fn_name, inputs, outputs, timeout)`	function	Execute and score the code

Table 6.1: `evaluator.py`: all items.

6.3 Full Source Code

Listing 6.1: `evaluator.py` — complete source.

```python
"""
evaluator.py -- Code extraction and sandboxed test-case execution.

Execution model (Codewars APPS fn_name problems):
    inputs[i]  = Python list already parsed, e.g. [1] or ["hello", 2]
    outputs[i] = [expected_return_value], e.g. [5] or [True]

    result = fn_name(*inputs[i])
    passed = ([result] == outputs[i])

Fitness = (test cases passed) / (total test cases)   -- partial credit.
"""

import re
import subprocess
import sys
from typing import Optional
```

```python


# ---------------------------
# Code extraction
# ---------------------------

def extract_code(response: str) -> str:
    """
    Pull the Python code block out of an LLM response.
    Tries ```python ... ``` first, then any ``` ... ```, then the raw text.
    """
    # Markdown fenced code block with python tag
    m = re.search(r"```python\s*\n([\s\S]*?)```", response)
    if m:
        return m.group(1).strip()

    # Any fenced code block
    m = re.search(r"```\s*\n?([\s\S]*?)```", response)
    if m:
        return m.group(1).strip()

    # No fences -- assume the whole response is code
    return response.strip()


def has_function(code: str, fn_name: str) -> bool:
    """Return True if `def fn_name(` appears in the code."""
    return bool(re.search(rf"\bdef\s+{re.escape(fn_name)}\s*\(", code))


# ---------------------------
# Subprocess test runner
# ---------------------------

_HARNESS_TEMPLATE = """\
import sys
sys.set_int_max_str_digits(1_000_000)

{code}

def __run__(inp, expected):
    try:
        result = {fn_name}(*inp)
        sys.stdout.write("__R__:" + ("1" if [result] == expected else "0") + "\\n")
        sys.stdout.flush()
    except Exception:
        sys.stdout.write("__R__:0\\n")
        sys.stdout.flush()

{calls}
"""


```

```python
def _build_script(code: str, fn_name: str, inputs: list, outputs: list) -> str:
    """Build the standalone test script that will be executed in a subprocess."""
    call_lines = "\n".join(
        f"__run__({repr(inp)}, {repr(out)})"
        for inp, out in zip(inputs, outputs)
    )
    return _HARNESS_TEMPLATE.format(
        code=code,
        fn_name=fn_name,
        calls=call_lines,
    )


def run_tests(
    code: str,
    fn_name: str,
    inputs: list,
    outputs: list,
    timeout: int = 5,
) -> float:
    """
    Execute generated code against all test cases in an isolated subprocess.

    Returns partial-credit score: (passed tests) / (total tests).
    Returns 0.0 on any execution error, timeout, or missing function.
    """
    if not code or not has_function(code, fn_name):
        return 0.0

    script = _build_script(code, fn_name, inputs, outputs)

    try:
        proc = subprocess.run(
            [sys.executable, "-c", script],
            capture_output=True,
            text=True,
            timeout=timeout,
        )
        # Parse only our sentinel lines to ignore any stray print() in generated code
        results = [
            int(line[len("__R__:"):])
            for line in proc.stdout.splitlines()
            if line.startswith("__R__:")
        ]
        if not results:
            return 0.0
        return sum(results) / len(inputs)   # partial credit over ALL test cases

    except subprocess.TimeoutExpired:
        return 0.0
    except Exception:
        return 0.0
```

6.4 Function-by-Function Analysis

6.4.1 extract_code(response)

extract_code(response: str) -> str

Defined: evaluator.py, line 24
Called by: ga.compute_fitness() [ga.py:68]; main.evaluate_on_holdout() [main.py:54]
Calls: re.search() (twice at most)
Parameters:
response — raw text string returned by the LLM (may contain markdown)
Returns: str — Python code only, whitespace-stripped

LLMs frequently wrap code in Markdown fences. The function tries three strategies in order, returning as soon as one succeeds:

Strategy 1 — tagged python fence (lines 29–31):

```
# Regex: r"```python\s*\n([\s\S]*?)```"
# Matches:  ```python
#           <any content, including newlines ([\s\S]*?)>
#           ```
# Group 1 = the code inside the fences (captured by the first parentheses)
```

Strategy 2 — untagged fence (lines 33–35):

```
# Regex: r"```\s*\n?([\s\S]*?)```"
# Matches: ``` (with optional whitespace / newline before content)
# Handles responses like:
#   ```
#   def foo(): ...
#   ```
```

Strategy 3 — raw text (line 38): If no fences are found, assume the entire response is already plain code (some prompts cause the LLM to output code directly with no markdown).

Worked Example — extract_code examples

```
# -- Strategy 1 fires --
response = "Here is my solution:\n```python\ndef nth_char(words):\n    return
    ''.join(w[i] for i, w in enumerate(words))\n```"
extract_code(response)
# => "def nth_char(words):\n    return ''.join(w[i] for i, w in enumerate(
    words))"

# -- Strategy 2 fires --
response = "```\ndef nth_char(words):\n    return ''.join(w[i] for i, w in
    enumerate(words))\n```"
extract_code(response)
# => "def nth_char(words):\n    return ''.join(w[i] for i, w in enumerate(
    words))"

# -- Strategy 3 fires --
response = "def nth_char(words):\n    return ''.join(w[i] for i, w in
    enumerate(words))"
extract_code(response)
```

```
# => "def nth_char(words):\n    return ''.join(w[i] for i, w in enumerate(
    words))"
```

6.4.2 has_function(code, fn_name)

has_function(code: str, fn_name: str) -> bool

Defined: evaluator.py, line 43
Called by: run_tests() [evaluator.py:97]
Calls: re.search(), re.escape()
Parameters:
code — extracted Python code string
fn_name — the function name the LLM was asked to write (e.g., "nth_char")
Returns: bool — True if def fn_name(appears in *code*

```
# Regex: rf"\bdef\s+{re.escape(fn_name)}\s*\("
#
# \b          = word boundary (prevents matching "def foo_bar(" when fn_name="
    bar")
# def         = literal keyword
# \s+         = one or more spaces/tabs
# fn_name     = exact function name (re.escape handles special chars like ".")
# \s*         = zero or more spaces before "("
# \(          = opening parenthesis

has_function("def nth_char(words):\n    return ...", "nth_char")  # True
has_function("def nth_char(words):\n    return ...", "sum_list")  # False
has_function("# nth_char defined elsewhere", "nth_char")          # False (no "
    def ")
has_function("class Solution:\n    def nth_char(self): ...", "nth_char") # True
```

If has_function returns False, run_tests immediately returns 0.0 without attempting execution. This guards against the LLM writing the wrong function name, which would cause an unresolvable NameError in the subprocess.

6.4.3 _HARNESS_TEMPLATE

_HARNESS_TEMPLATE: str (module-level constant)

Defined: evaluator.py, line 52
Used by: _build_script() to produce the complete subprocess script
Template variables: {code}, {fn_name}, {calls}

The template defines a small Python program that will be run in a subprocess:

```
# _HARNESS_TEMPLATE (with placeholders shown):
import sys
sys.set_int_max_str_digits(1_000_000)  # same large-int fix as dataset.py

{code}                      # <-- the LLM-generated function pasted verbatim

def __run__(inp, expected):
    try:
        result = {fn_name}(*inp)     # <-- calls the generated function
        # [result] wraps return value in list to match APPS output format
```

```
        sys.stdout.write("__R__:" + ("1" if [result] == expected else "0") + "\n
            ")
        sys.stdout.flush()
    except Exception:                # <-- catches ALL exceptions (NameError,
        TypeError...)
        sys.stdout.write("__R__:0\n")
        sys.stdout.flush()

{calls}     # <-- one __run__(...) call per test case
```

Why [result] == expected?

Codewars APPS problems store expected outputs as `[return_value]` (a list containing the single return value). For example, if `nth_char` should return `"oes"`, then `outputs[i] = ["oes"]`. Wrapping the actual result in `[result]` makes the comparison format match: `["oes"] == ["oes"]` is `True`; `["oes"] == ["abc"]` is `False`.

Without the wrapping: `"oes" == ["oes"]` would always be `False` even for correct code.

Why a subprocess?

The LLM-generated code is untrusted. Importing it directly into the GA process could:

- Crash the entire GA (via a top-level `sys.exit()` or segfault).
- Corrupt global state (e.g., monkey-patching Python builtins).
- Hang indefinitely (infinite loop with no timeout mechanism).

Running in a subprocess isolates all of this: if the subprocess crashes, hangs, or produces garbage, the GA process is completely unaffected.

6.4.4 `_build_script(code, fn_name, inputs, outputs)`

`_build_script(code, fn_name, inputs, outputs) -> str`

Defined: `evaluator.py`, line 71
Called by: `run_tests()` [evaluator.py:100]
Calls: `repr()`, `str.join()`, `_HARNESS_TEMPLATE.format()`
Parameters:
code — extracted Python source string
fn_name — function name the harness will call
inputs — list of test-case argument lists
outputs — list of `[expected_return_value]` lists
Returns: `str` — complete, executable Python script

How it builds the `calls` section:

```
inputs  = [[1, 2, 3], [0], []]
outputs = [[6],       [0], [0]]

call_lines = "\n".join(
    f"__run__({repr(inp)}, {repr(out)})"
    for inp, out in zip(inputs, outputs)
)
# call_lines =
# __run__([1, 2, 3], [6])
# __run__([0], [0])
# __run__([], [0])
```

Worked Example — Complete script for a sum_list problem

```
# Inputs to _build_script:
code    = "def sum_list(nums):\n    return sum(nums)"
fn_name = "sum_list"
inputs  = [[1, 2, 3], [0], [], [-1, 1]]
outputs = [[6],       [0], [0], [0]]

# Script produced by _build_script:
"""
import sys
sys.set_int_max_str_digits(1_000_000)

def sum_list(nums):
    return sum(nums)

def __run__(inp, expected):
    try:
        result = sum_list(*inp)
        sys.stdout.write("__R__:" + ("1" if [result] == expected else "0") +
            "\n")
        sys.stdout.flush()
    except Exception:
        sys.stdout.write("__R__:0\n")
        sys.stdout.flush()

__run__([1, 2, 3], [6])
__run__([0], [0])
__run__([], [0])
__run__([-1, 1], [0])
"""
# When run:
# sum_list(1, 2, 3) => sum([1,2,3]) => 6;  [6] == [6]  => __R__:1
# sum_list(0)       => sum([0]) => 0;      [0] == [0]  => __R__:1
# sum_list()        => sum([]) => 0;       [0] == [0]  => __R__:1
# sum_list(-1, 1)   => sum([-1,1]) => 0;   [0] == [0]  => __R__:1
# stdout: "__R__:1\n__R__:1\n__R__:1\n__R__:1\n"
```

6.4.5 run_tests(code, fn_name, inputs, outputs, timeout=5)

run_tests(code, fn_name, inputs, outputs, timeout=5) -> float

Defined: `evaluator.py`, line 84
Called by: `ga.compute_fitness()` [ga.py:69]; `main.evaluate_on_holdout()` [main.py:56]
Calls: `has_function()`, `_build_script()`, `subprocess.run()`
Parameters:
code — extracted Python code string
fn_name — expected function name (e.g., `"sum_list"`)
inputs — list of argument-list test cases
outputs — list of `[expected]` test cases
timeout — subprocess kill timeout in seconds (default: 5)
Returns: `float` in `[0.0, 1.0]` — fraction of test cases passed

Step-by-step walkthrough:

1. **Lines 97–98**: Early return 0.0 if *code* is empty (the LLM returned nothing) or if `has_function` returns False (the required function is not defined).

2. **Lines 100**: Call `_build_script` to assemble the executable test script.

3. **Lines 102–108**: Launch the script in an isolated subprocess:

```
proc = subprocess.run(
    [sys.executable, "-c", script],  # python3 -c "..."  (full script as
        argument)
    capture_output=True,              # capture stdout and stderr
    text=True,                        # decode bytes to str automatically
    timeout=timeout,                  # kill process after 5 seconds
)
```

4. **Lines 109–114**: Parse `proc.stdout` for sentinel lines:

```
results = [
    int(line[len("__R__:"):])          # "1" or "0" after the prefix
    for line in proc.stdout.splitlines()
    if line.startswith("__R__:")       # ignore all other lines (stray
        prints)
]
# If stdout = "__R__:1\n__R__:1\n__R__:0\n__R__:1\n"
# results  = [1, 1, 0, 1]
```

5. **Lines 115–116**: If `results` is empty (script crashed on load — e.g. syntax error), return 0.0.

6. **Line 117**: Return partial-credit score:

```
return sum(results) / len(inputs)
#          ^-- passed tests          ^-- TOTAL test cases (not just those
    that ran)

# Example: sum([1,1,0,1]) / 4 = 3/4 = 0.75
# If script crashes after 2 tests: results=[1,1], len(inputs)=4 => 0.5
# (not 1.0, because the 2 un-run tests count as failures in the denominator
    )
```

7. **Lines 118–121**: Exception handling:
   - `subprocess.TimeoutExpired` — the 5-second limit was hit; return 0.0.
   - Any other exception — subprocess module error; return 0.0.

Division by `len(inputs)`, not `len(results)`

The denominator is `len(inputs)` (total test cases) not `len(results)` (test cases that produced output). If generated code crashes midway through the tests, the un-run tests count as failures. This correctly penalises fragile code that only works on some inputs before crashing.

The Sentinel Design

Generated code may contain stray `print()` statements (the LLM might print debug output like "Input is 5"). These would mix into `proc.stdout` and corrupt the results if we tried to parse every line. By only looking at lines that start with the unique prefix `__R__:`, we ignore all other output completely. The prefix was chosen to be unlikely to appear in any realistic program output.

Chapter 7

Cost Tracking: cost_tracker.py

7.1 Purpose

Every call to the Claude API costs real money. This module logs the exact token count of every call, computes the dollar cost using official Anthropic pricing, aggregates costs by phase (fitness, crossover, etc.) and by model, and saves a timestamped JSON report to `Code/Costs/` after each run.

Costs folder location (updated)

Cost reports are saved to `Code/Costs/cost_TIMESTAMP.json` — inside the same `Code/` directory as the source files. The constant `COSTS_DIR` is set as:
`COSTS_DIR = Path(__file__).parent / "Costs"`
which resolves to `/home/.../Code/Costs/`.

7.2 File at a Glance

Item	Type	Description
`PRICING`	`dict`	Official Anthropic prices per model (input/output $/MTok)
`COSTS_DIR`	`Path`	Output directory: `Code/Costs/`
`CallRecord`	dataclass	One API call's metadata and cost
`CostTracker`	class	Accumulates records; writes JSON report
`tracker`	`CostTracker`	Module-level singleton imported by `llm.py`

Table 7.1: `cost_tracker.py`: all exported items.

7.3 Full Source Code

Listing 7.1: `cost_tracker.py` — complete source.

```python
"""
cost_tracker.py -- Track every API call's token usage and compute
    running cost.
"""

import json
import time
from dataclasses import dataclass, field
from datetime import datetime
from pathlib import Path

# Official Anthropic pricing (USD per 1 million tokens) -- April 2026
```

```python
PRICING = {
    "claude-haiku-4-5-20251001": (1.00,  5.00),
    "claude-haiku-3-5-20241022": (0.80,  4.00),
    "claude-haiku-3-20240307":   (0.25,  1.25),
    "claude-sonnet-4-6":         (3.00, 15.00),
    "claude-sonnet-4-5-20250514":(3.00, 15.00),
    "claude-sonnet-4-20250514":  (3.00, 15.00),
    "claude-opus-4-6":           (5.00, 25.00),
    # Batch API (50% discount)
    "claude-haiku-4-5-20251001:batch": (0.50, 2.50),
    "claude-sonnet-4-6:batch":         (1.50, 7.50),
}

COSTS_DIR = Path(__file__).parent / "Costs"


@dataclass
class CallRecord:
    phase: str          # "fitness", "crossover", "mutate_inject", etc.
    model: str
    input_tokens: int
    output_tokens: int

    @property
    def input_cost(self) -> float:
        rate_in, _ = PRICING.get(self.model, (0, 0))
        return self.input_tokens * rate_in / 1_000_000

    @property
    def output_cost(self) -> float:
        _, rate_out = PRICING.get(self.model, (0, 0))
        return self.output_tokens * rate_out / 1_000_000

    @property
    def total_cost(self) -> float:
        return self.input_cost + self.output_cost


class CostTracker:
    """Accumulates CallRecords and saves a full breakdown JSON on flush."""

    def __init__(self):
        self.calls: list[CallRecord] = []
        self.start_time: float = time.time()

    def record(self, phase: str, model: str,
               input_tokens: int, output_tokens: int):
        """Log one API call."""
        self.calls.append(CallRecord(
            phase=phase, model=model,
            input_tokens=input_tokens,
            output_tokens=output_tokens,
        ))

    def _by_phase(self) -> dict:
```

```
        phases: dict[str, dict] = {}
        for c in self.calls:
            if c.phase not in phases:
                phases[c.phase] = {
                    "calls": 0, "input_tokens": 0, "output_tokens": 0,
                    "input_cost_usd": 0.0, "output_cost_usd": 0.0,
                    "total_cost_usd": 0.0,
                }
            p = phases[c.phase]
            p["calls"]            += 1
            p["input_tokens"]     += c.input_tokens
            p["output_tokens"]    += c.output_tokens
            p["input_cost_usd"]   += c.input_cost
            p["output_cost_usd"]  += c.output_cost
            p["total_cost_usd"]   += c.total_cost
        for p in phases.values():
            for k in ("input_cost_usd", "output_cost_usd", "total_cost_usd"):
                p[k] = round(p[k], 6)
        return phases

    def _by_model(self) -> dict:
        models: dict[str, dict] = {}
        for c in self.calls:
            if c.model not in models:
                models[c.model] = {
                    "calls": 0, "input_tokens": 0, "output_tokens": 0,
                    "input_cost_usd": 0.0, "output_cost_usd": 0.0,
                    "total_cost_usd": 0.0,
                }
            m = models[c.model]
            m["calls"]            += 1
            m["input_tokens"]     += c.input_tokens
            m["output_tokens"]    += c.output_tokens
            m["input_cost_usd"]   += c.input_cost
            m["output_cost_usd"]  += c.output_cost
            m["total_cost_usd"]   += c.total_cost
        for m in models.values():
            for k in ("input_cost_usd", "output_cost_usd", "total_cost_usd"):
                m[k] = round(m[k], 6)
        return models

    def total(self) -> float:
        return sum(c.total_cost for c in self.calls)

    def flush(self, extra: dict | None = None) -> Path:
        """Write a timestamped JSON to the Costs/ folder."""
        COSTS_DIR.mkdir(parents=True, exist_ok=True)
        ts = datetime.now().strftime("%Y%m%d_%H%M%S")
        path = COSTS_DIR / f"cost_{ts}.json"

        total_in  = sum(c.input_tokens for c in self.calls)
        total_out = sum(c.output_tokens for c in self.calls)
        elapsed   = time.time() - self.start_time

        report = {
```

```
122             "timestamp":        ts,
123             "elapsed_seconds":  round(elapsed, 1),
124             "total_api_calls":  len(self.calls),
125             "total_input_tokens":  total_in,
126             "total_output_tokens": total_out,
127             "total_input_cost_usd":  round(sum(c.input_cost  for c in
                    self.calls), 6),
128             "total_output_cost_usd": round(sum(c.output_cost for c in
                    self.calls), 6),
129             "grand_total_usd":       round(self.total(), 6),
130             "breakdown_by_phase": self._by_phase(),
131             "breakdown_by_model": self._by_model(),
132             "pricing_used": {
133                 model: {"input_per_MTok": r[0], "output_per_MTok": r
                        [1]}
134                 for model, r in PRICING.items()
135                 if not model.endswith(":batch")
136             },
137         }
138         if extra:
139             report["run_info"] = extra
140
141         path.write_text(json.dumps(report, indent=2))
142         return path
143
144
145 # Module-level singleton imported by llm.py
146 tracker = CostTracker()
```

7.4 Component-by-Component Analysis

7.4.1 The PRICING Dictionary

PRICING: dict[str, tuple[float, float]]

Defined: cost_tracker.py, line 12
Used by: CallRecord.input_cost, CallRecord.output_cost, CostTracker.flush() (in the pricing_used section)
Format: model_id: (input_$/MTok, output_$/MTok)

"MTok" means million tokens. Pricing formula:

$$cost = \frac{input_tokens \times rate_in}{1{,}000{,}000} + \frac{output_tokens \times rate_out}{1{,}000{,}000}$$

Worked Example — Cost for one typical fitness call

```
# A typical llm.generate_code() call on Haiku 4.5:
model        = "claude-haiku-4-5-20251001"
input_tokens = 450    # system prompt (150) + problem question (300)
output_tokens = 80    # the generated function (~60 lines of Python)

# From PRICING: rate_in=1.00, rate_out=5.00  (USD per million tokens)
input_cost  = 450  * 1.00 / 1_000_000 = $0.000450
output_cost =  80  * 5.00 / 1_000_000 = $0.000400
```

```
total_cost  = $0.000850  per call

# Over 27,700 fitness calls in a full run:
# estimated_fitness_cost = 27700 * $0.000850 ~= $23.54
```

7.4.2 CallRecord Dataclass

@dataclass class CallRecord

Defined: cost_tracker.py, line 27
Created by: CostTracker.record() [cost_tracker.py:55]
Fields:
phase: str — which GA operation (e.g., "fitness", "crossover", "mutate_inject")
model: str — Claude model ID
input_tokens: int — tokens in the prompt sent to the API
output_tokens: int — tokens in the API's response
Computed properties:
input_cost: float — $ cost for input tokens
output_cost: float — $ cost for output tokens
total_cost: float — input_cost + output_cost

The three @property methods compute cost on the fly from the PRICING dict — they are not stored fields, so there is no risk of the stored cost getting out of sync if PRICING changes.

7.4.3 CostTracker Class

class CostTracker

Defined: cost_tracker.py, line 48
Instantiated as: module-level singleton tracker (line 178)
Imported by: llm.py via from cost_tracker import tracker

CostTracker.record(phase, model, input_tokens, output_tokens)

```
Called by: llm._call()  [llm.py:54-58]
  Every time the Claude API responds successfully, _call() calls:
    tracker.record(
        phase=phase,                          # e.g., "fitness"
        model=model,                          # e.g., "claude-haiku
            -4-5-20251001"
        input_tokens=resp.usage.input_tokens,  # from API response object
        output_tokens=resp.usage.output_tokens # from API response object
    )
  This appends a new CallRecord to self.calls (the in-memory log).
```

CostTracker._by_phase() and CostTracker._by_model()

Both aggregate the self.calls list into a summary dict. _by_phase() groups by CallRecord.phase; _by_model() groups by CallRecord.model.

Worked Example — Sample _by_phase() output

```
{
    "fitness": {
        "calls": 27700,
        "input_tokens": 12465000,
        "output_tokens": 2216000,
        "input_cost_usd": 12.465,
        "output_cost_usd": 11.08,
        "total_cost_usd": 23.545
    },
    "crossover": {
        "calls": 208,
        "input_tokens": 145600,
        "output_tokens": 74880,
        "input_cost_usd": 0.4368,
        "output_cost_usd": 1.1232,
        "total_cost_usd": 1.56
    },
    "mutate_inject":   {...},
    "mutate_delete":   {...},
    "mutate_rephrase": {...}
}
```

CostTracker.flush(extra=None)

flush(extra: dict | None = None) -> Path

Called by: `main.main()` [main.py:154]
Calls: `COSTS_DIR.mkdir()`, `self._by_phase()`, `self._by_model()`, `path.write_text()`, `json.dumps()`
Returns: `Path` — path to the written JSON file

Workflow:

1. Creates `Code/Costs/` directory if it does not exist.
2. Generates a timestamp string (e.g., `"20260417_142035"`).
3. Assembles a report dict with summary totals, per-phase breakdown, per-model breakdown, and the pricing table used.
4. If *extra* is provided (e.g., best fitness scores), adds it under `"run_info"`.
5. Writes the report as formatted JSON and returns the path.

7.4.4 The Singleton Pattern

```python
# At the bottom of cost_tracker.py (line 178):
tracker = CostTracker()

# In llm.py (line 15):
from cost_tracker import tracker    # imports the SAME object

# Python caches imported modules.  When llm.py does "from cost_tracker import
    tracker",
# it gets the exact same CostTracker instance that was created when cost_tracker
    .py
# was first imported.  So every _call() in llm.py adds to the same list.

# Later in main.py (line 23):
```

```python
from cost_tracker import tracker   # SAME object again

# When main.main() calls tracker.flush(), it sees ALL calls made by llm.py.
# No data is lost between modules.
```

Chapter 8

LLM Interface: `llm.py`

8.1 Purpose

This is the *only* file in the entire project that communicates with the Anthropic API. All other files are either pure Python computation or file I/O. The file provides five public functions, all of which flow through a single private `_call()` wrapper that handles authentication, retries, and cost logging.

8.2 File at a Glance

Item	Type	Description
`_client`	global var	Cached `anthropic.Anthropic` instance (lazy init)
`_get_client()`	private func	Initialise and return the client (once)
`_call(...)`	private func	Core API wrapper: retry + cost logging
`generate_code()`	public func	Ask target LLM to write code (fitness)
`crossover()`	public func	Merge two parent prompts into one offspring
`mutate_inject()`	public func	Add a new strategy sentence to a prompt
`mutate_delete()`	public func	Remove the weakest sentence from a prompt
`mutate_rephrase()`	public func	Rephrase a prompt without changing meaning
`_CROSSOVER_SYSTEM`	str	System prompt for the crossover meta-call
`_CROSSOVER_USER`	str	User prompt template for crossover
`_MUT_SYSTEM`	str	System prompt for all three mutation meta-calls

Table 8.1: `llm.py`: all items.

8.3 Full Source Code

Listing 8.1: `llm.py` — complete source.

```
"""
llm.py -- All Claude API calls: code generation, crossover, and mutation.

Two models are used:
    TARGET_MODEL    -- generates code (cheap, deterministic, temperature 0)
    OPTIMIZER_MODEL -- performs crossover and mutation (creative, temperature 0.7)

```

```python
Every call is logged to the CostTracker singleton (see cost_tracker.py).
"""

import re
import time
import anthropic
import config
from cost_tracker import tracker


_client: anthropic.Anthropic | None = None


def _get_client() -> anthropic.Anthropic:
    """Lazy-initialise the Anthropic client."""
    global _client
    if _client is None:
        _client = anthropic.Anthropic(api_key=config.ANTHROPIC_API_KEY)
    return _client


def _call(
    phase: str,
    model: str,
    temperature: float,
    max_tokens: int,
    system: str,
    user: str,
    retries: int = 3,
) -> str:
    """
    Call the Claude API with automatic retry on transient errors.
    Logs token usage to the global CostTracker.
    Returns the assistant's text response.
    """
    client = _get_client()
    for attempt in range(retries):
        try:
            resp = client.messages.create(
                model=model,
                max_tokens=max_tokens,
                temperature=temperature,
                system=system,
                messages=[{"role": "user", "content": user}],
            )
            # Log tokens to cost tracker
            tracker.record(
                phase=phase,
                model=model,
                input_tokens=resp.usage.input_tokens,
                output_tokens=resp.usage.output_tokens,
            )
            return resp.content[0].text
        except anthropic.RateLimitError:
            wait = 2 ** attempt * 5
```

```
            print(f"  [rate-limit] waiting {wait}s ...")
            time.sleep(wait)
        except anthropic.APIError as e:
            if attempt == retries - 1:
                print(f"  [api-error] {e}")
                return ""
            time.sleep(2)
    return ""


# --------------------------
# Code generation  (fitness evaluation)
# --------------------------

def generate_code(system_prompt: str, question: str, fn_name: str) -> str:
    """
    Ask the target LLM to solve a coding problem.
    Returns the raw LLM response (code extraction happens in evaluator.py).
    """
    user_msg = (
        f"{question}\n\n"
        f"Write a Python function named `{fn_name}`."
    )
    return _call(
        phase="fitness",
        model=config.TARGET_MODEL,
        temperature=config.TARGET_TEMPERATURE,
        max_tokens=config.MAX_CODE_TOKENS,
        system=system_prompt,
        user=user_msg,
    )


# --------------------------
# Crossover  (LLM-mediated merge of two parent prompts)
# --------------------------

_CROSSOVER_SYSTEM = (
    "You merge coding-instruction prompts. Output ONLY the merged prompt -- "
    "no commentary, no labels, no markdown."
)

_CROSSOVER_USER = """\
Combine these two coding-instruction prompts into one coherent prompt.
Preserve the strongest strategies from each parent (e.g. edge-case thinking
from one, algorithm identification from the other).
Do not simply concatenate -- produce a unified instruction paragraph.

-- Parent A --
{parent_a}

-- Parent B --
{parent_b}
```

```python
"""


def crossover(prompt_a: str, prompt_b: str) -> str:
    """Merge two parent prompts into one offspring via the optimizer LLM."""
    return _call(
        phase="crossover",
        model=config.OPTIMIZER_MODEL,
        temperature=config.OPTIMIZER_TEMPERATURE,
        max_tokens=config.MAX_OPERATOR_TOKENS,
        system=_CROSSOVER_SYSTEM,
        user=_CROSSOVER_USER.format(parent_a=prompt_a, parent_b=prompt_b),
    ).strip()


# ---------------------------
# Mutation  (three independent operators)
# ---------------------------

_MUT_SYSTEM = (
    "You edit coding-instruction prompts. Output ONLY the modified prompt -- "
    "no commentary, no labels, no markdown."
)


def mutate_inject(prompt: str) -> str:
    """Add one new instructional strategy to the prompt."""
    user = (
        "Add one new, specific coding strategy to this instruction prompt. "
        "Examples of strategies: 'validate your output against the examples', "
        "'consider time complexity', 'check for off-by-one errors'. "
        "Output only the modified prompt.\n\n"
        f"Prompt:\n{prompt}"
    )
    return _call(
        phase="mutate_inject",
        model=config.OPTIMIZER_MODEL,
        temperature=config.OPTIMIZER_TEMPERATURE,
        max_tokens=config.MAX_OPERATOR_TOKENS,
        system=_MUT_SYSTEM,
        user=user,
    ).strip()


def mutate_delete(prompt: str) -> str:
    """Remove one sentence from the prompt to make it leaner."""
    sentences = re.split(r"(?<=[.!?])\s+", prompt)
    if len(sentences) <= 1:
        return prompt  # nothing to delete -- keep as-is
    user = (
        "Remove one sentence from this instruction prompt -- the one that "
```

```
        "contributes the least to code-generation quality. Keep everything "
        "else verbatim. Output only the modified prompt.\n\n"
        f"Prompt:\n{prompt}"
    )
    return _call(
        phase="mutate_delete",
        model=config.OPTIMIZER_MODEL,
        temperature=config.OPTIMIZER_TEMPERATURE,
        max_tokens=config.MAX_OPERATOR_TOKENS,
        system=_MUT_SYSTEM,
        user=user,
    ).strip()


def mutate_rephrase(prompt: str) -> str:
    """Rephrase the prompt without changing its meaning."""
    user = (
        "Rephrase this instruction prompt using different words, but keep "
        "the same meaning and all the same strategies. "
        "Output only the rephrased prompt.\n\n"
        f"Prompt:\n{prompt}"
    )
    return _call(
        phase="mutate_rephrase",
        model=config.OPTIMIZER_MODEL,
        temperature=config.OPTIMIZER_TEMPERATURE,
        max_tokens=config.MAX_OPERATOR_TOKENS,
        system=_MUT_SYSTEM,
        user=user,
    ).strip()
```

8.4 Function-by-Function Analysis

8.4.1 _get_client()

_get_client() -> anthropic.Anthropic

Defined: llm.py, line 21
Called by: _call() [llm.py:43]
Calls: anthropic.Anthropic(api_key=...)
Returns: The module-level cached anthropic.Anthropic instance

This function implements the **lazy initialisation** pattern: the anthropic.Anthropic client is not created until the first API call is made. This avoids crashing on import if the API key is missing.

```
# The global is None at module load time:
_client: anthropic.Anthropic | None = None

# First call to _get_client():
#   _client is None  =>  create it, store it, return it
_client = anthropic.Anthropic(api_key="sk-ant-...")
```

```
# All subsequent calls:
#   _client is not None  =>  return the cached object immediately
#   (the Anthropic SDK handles connection pooling internally)
```

8.4.2 _call(phase, model, temperature, max_tokens, system, user, retries=3)

_call(...) -> str

Defined: llm.py, line 29
Called by: generate_code(), crossover(), mutate_inject(), mutate_delete(), mutate_rephrase()
Calls: _get_client(), client.messages.create(), tracker.record(), time.sleep()

Parameters:
phase — label for cost tracking (e.g., "fitness", "crossover")
model — Claude model ID (e.g., "claude-haiku-4-5-20251001")
temperature — 0.0 for deterministic, 0.7 for creative
max_tokens — response token limit (1024 for code, 600 for operators)
system — the system prompt (the GA chromosome for fitness; meta-prompt for operators)
user — the user message (question + fn_name for fitness; prompts for operators)
retries — number of attempts before giving up (default: 3)

Returns: str — assistant's text, or "" if all retries fail

Step-by-step walkthrough:

1. **Line 43**: Get the Anthropic client (cached after first call).
2. **Line 44**: Loop up to *retries*=3 times (attempts 0, 1, 2).
3. **Lines 46–52**: Call the Claude API:

```
resp = client.messages.create(
    model=model,                # which Claude model
    max_tokens=max_tokens,      # cap on response length
    temperature=temperature,    # 0.0=same output every time; 0.7=varied
    system=system,              # the system prompt (before the conversation)
    messages=[{"role": "user", "content": user}]  # one-turn conversation
)
# resp.content[0].text = the assistant's text response
# resp.usage.input_tokens  = tokens consumed by system + user
# resp.usage.output_tokens = tokens generated in response
```

4. **Lines 53–58**: Log the call to the cost tracker singleton:

```
tracker.record(
    phase=phase,                            # e.g., "fitness"
    model=model,
    input_tokens=resp.usage.input_tokens,
    output_tokens=resp.usage.output_tokens,
)
```

5. **Line 60**: Return resp.content[0].text — the text of the first content block (Claude responses always have at least one text block).
6. **Lines 61–63**: anthropic.RateLimitError handler — the API returned HTTP 429 (too many requests). Wait with **exponential backoff**:

```
wait = 2 ** attempt * 5
# attempt 0: wait = 1 * 5 = 5 seconds
# attempt 1: wait = 2 * 5 = 10 seconds
# attempt 2: wait = 4 * 5 = 20 seconds
# After 3 attempts: return ""  (the while loop ends)
```

7. **Lines 64–68**: `anthropic.APIError` handler — any other API error (server error, malformed request, etc.).
   - If this is the last attempt (`attempt == retries - 1`): print the error and return `""`.
   - Otherwise: wait 2 seconds and try again.
8. **Line 70**: If the loop exits without returning (should not normally happen), return `""` as a safety net.

What happens when _call returns ""?

If `_call` returns an empty string, the caller (e.g. `generate_code`) passes it to `evaluator.extract_code("")` which returns `""`, then `evaluator.run_tests("")` returns 0.0 immediately (the empty-code guard). The individual gets fitness 0.0 for that problem. The run continues — a failed API call does *not* crash the GA.

8.4.3 `generate_code(system_prompt, question, fn_name)`

`generate_code(system_prompt, question, fn_name) -> str`

Defined: `llm.py`, line 77
Called by: `ga.compute_fitness()` [ga.py:63]; `main.evaluate_on_holdout()` [main.py:52]
Calls: `_call(phase="fitness", model=TARGET_MODEL, temperature=0.0, ...)`

Parameters:
system_prompt — the GA chromosome (the prompt being evolved)
question — APPS problem text (from `prob["question"]`)
fn_name — function name the LLM must write (from `prob["fn_name"]`)

Returns: `str` — raw LLM text (NOT extracted code; that happens in evaluator)

```
# How the messages are assembled:

system = system_prompt
       = "Think like a competitive programmer. Identify the algorithm pattern
          ..."
         (the chromosome being evaluated)

user   = f"{question}\n\nWrite a Python function named `{fn_name}`."
       = "Complete the function that takes an array of words.\n"
         "You must concatenate the nth letter from each word...\n\n"
         "Write a Python function named `nth_char`."

# The separation is intentional:
# - system = GENERAL instruction (applies to ALL problems)
# - user   = SPECIFIC problem (changes per problem)
# This allows the LLM to apply the strategy from the system prompt
# to the specific problem in the user message.
```

Worked Example — generate_code call and return

```
response = llm.generate_code(
    system_prompt="Think like a competitive programmer. Identify the
        algorithm "
                  "pattern (math, string manipulation, recursion, sorting,
                      hashing). "
                  "Write the most direct, correct Python solution. Output
                      only the "
                  "function definition.",
    question="Complete the function that takes an array of words.\n"
             "You must concatenate the nth letter from each word using the "
             "enumerating order of the word.\n"
             "Example: nth_char(['yoda','best','has']) == 'oes'",
    fn_name="nth_char"
)

# A typical LLM response (raw text):
response = """```python
def nth_char(words):
    return ''.join(word[i] for i, word in enumerate(words))
```"""

This raw response is then passed to evaluator.extract_code(response)
which returns: "def nth_char(words):\n   return ''.join(word[i] for i,
    word in enumerate(words))"
```

### 8.4.4 crossover(prompt_a, prompt_b)

**crossover(prompt_a: str, prompt_b: str) -> str**

**Defined:** `llm.py`, line 125
**Called by:** `ga.breed()` [ga.py:117] when `rng.random() < 0.8`
**Calls:** `_call(phase="crossover", model=OPTIMIZER_MODEL, temperature=0.7, max_tokens=600, ...)`

**Parameters:**
*prompt_a* — first parent's prompt string
*prompt_b* — second parent's prompt string

**Returns:** `str` — merged offspring prompt, whitespace-stripped

The Optimizer LLM (Sonnet 4.6) receives a meta-prompt asking it to blend the two parents:

```
_CROSSOVER_SYSTEM:
"You merge coding-instruction prompts. Output ONLY the merged prompt --
 no commentary, no labels, no markdown."

_CROSSOVER_USER (with real parent prompts filled in):
"Combine these two coding-instruction prompts into one coherent prompt.
 Preserve the strongest strategies from each parent...
 Do not simply concatenate -- produce a unified instruction paragraph.

 -- Parent A --
 Think like a competitive programmer. Identify the algorithm pattern
 (math, string manipulation, recursion, sorting, hashing)...

 -- Parent B --
 You are a careful Python programmer. Explicitly think about boundary
```
```

```
 conditions: empty input, single element, zero, negative values..."

# Typical offspring output:
"Approach the problem like a competitive programmer: identify the
 algorithm pattern (math, recursion, string manipulation, sorting).
 Before coding, explicitly consider boundary conditions: empty input,
 zero, negative values, and single-element inputs. Write the most
 direct, correct Python solution and output only the function definition."
```

Crossover as semantic blending

Standard genetic algorithm crossover splits a binary string at a random point. Here, the "crossover" is an LLM call that *understands the meaning* of the two parent prompts and creates a semantically coherent child. The meta-prompt explicitly forbids simple concatenation — the LLM must integrate the strategies, resolving conflicts and removing redundancies.

8.4.5 mutate_inject(prompt)

mutate_inject(prompt: str) -> str

Defined: llm.py, line 147
Called by: ga.breed() [ga.py:124] when rng.random() < 0.25
Calls: _call(phase="mutate_inject", model=OPTIMIZER_MODEL, temperature=0.7, max_tokens=600, ...)

Returns: str — the same prompt with one new strategy sentence added

The injection meta-prompt:

```
"Add one new, specific coding strategy to this instruction prompt.
 Examples of strategies: 'validate your output against the examples',
 'consider time complexity', 'check for off-by-one errors'.
 Output only the modified prompt.

 Prompt:
 Think like a competitive programmer..."

# Example injected sentence: "After implementing, mentally trace through
# the examples to verify your solution produces the expected output."
```

8.4.6 mutate_delete(prompt)

mutate_delete(prompt: str) -> str

Defined: llm.py, line 166
Called by: ga.breed() [ga.py:127] when rng.random() < 0.15
Calls: re.split(), _call(...) (only if prompt has >1 sentence)

Returns: str — prompt with the weakest sentence removed (or unchanged if only 1 sentence)

Edge case guard:

```
sentences = re.split(r"(?<=[.!?])\s+", prompt)
# Split on whitespace that follows a sentence-ending punctuation.
# "Sentence one. Sentence two! Sentence three?"
# => ["Sentence one.", "Sentence two!", "Sentence three?"]
```

```
if len(sentences) <= 1:
    return prompt    # skip the API call; can't delete the only sentence
```

This check prevents calling the API for a one-sentence prompt where deletion would produce an empty string.

8.4.7 mutate_rephrase(prompt)

mutate_rephrase(prompt: str) -> str

Defined: llm.py, line 187
Called by: ga.breed() [ga.py:130] when rng.random() < 0.10
Calls: _call(phase="mutate_rephrase", model=OPTIMIZER_MODEL, temperature=0.7, max_tokens=600, ...)

Returns: str — same-meaning prompt in different words

```
# Input:
"Think like a competitive programmer. Identify the algorithm pattern..."

# Output (same meaning, different words):
"Approach the problem as a competitive coder would. Determine the
 underlying algorithmic pattern..."

# Purpose: even if the meaning is preserved, different wording may
# activate different associations in the target LLM (Haiku), potentially
# leading to better code generation on some problems.
```

Chapter 9

The Genetic Algorithm: `ga.py`

9.1 Purpose

This is the core of the project. It implements the complete evolutionary loop: initialise a population from seeds, evaluate fitness, select parents via tournament selection, create offspring via LLM-mediated crossover and mutation, re-evaluate offspring fitness, apply elitism, and repeat for 20 generations.

9.2 File at a Glance

Item	Type	Description
`Individual`	dataclass	One chromosome: prompt + fitness + lineage metadata
`compute_fitness()`	function	Evaluate one individual on all 100 evolution problems
`tournament_select()`	function	Pick the fittest of 3 random individuals
`breed()`	function	Crossover + mutation to produce one offspring
`run()`	function	The complete GA loop; returns best individual + history

Table 9.1: `ga.py`: all items.

9.3 Full Source Code

Listing 9.1: `ga.py` — complete source.

```python
"""
ga.py -- Genetic Algorithm for evolving system-level instruction prompts.

Pipeline summary (one generation):
    1. SELECTION  -- tournament (size 3), pick 2 parents
    2. CROSSOVER  -- LLM merges parents (rate 0.8)
    3. MUTATION   -- inject / delete / rephrase (independent)
    4. FITNESS    -- run code on 100 APPS problems
    5. REPLACEMENT -- generational with top-2 elitism
"""

from __future__ import annotations

import random
import uuid
```

```python
from dataclasses import dataclass, field

import config
import evaluator
import llm


# --------------------------
# Individual
# --------------------------

@dataclass
class Individual:
    """One chromosome = one system-level instruction prompt."""
    prompt: str
    fitness: float = -1.0
    uid: str = field(default_factory=lambda: uuid.uuid4().hex[:8])
    parent_a: str = ""       # uid of parent A (empty for seeds)
    parent_b: str = ""       # uid of parent B
    operators: str = ""      # operators applied, e.g. "crossover+inject"


# --------------------------
# Fitness
# --------------------------

def compute_fitness(individual: Individual, problems: list[dict]) -> float:
    """
    Evaluate a prompt on the evolution pool.
    Fitness = mean score across all problems (partial credit per problem).
    """
    total = 0.0
    for prob in problems:
        response = llm.generate_code(
            system_prompt=individual.prompt,
            question=prob["question"],
            fn_name=prob["fn_name"],
        )
        code = evaluator.extract_code(response)
        score = evaluator.run_tests(
            code=code,
            fn_name=prob["fn_name"],
            inputs=prob["inputs"],
            outputs=prob["outputs"],
            timeout=config.EXEC_TIMEOUT,
        )
        total += score
    return total / len(problems)


# --------------------------
# Selection
# --------------------------

```

```python
def tournament_select(population: list[Individual], rng: random.Random
    ) -> Individual:
    """Pick TOURNAMENT_SIZE random individuals, return the fittest."""
    contestants = rng.sample(population, config.TOURNAMENT_SIZE)
    return max(contestants, key=lambda ind: ind.fitness)


# --------------------------
# Crossover + mutation -- one offspring
# --------------------------

def breed(
    parent_a: Individual,
    parent_b: Individual,
    rng: random.Random,
) -> Individual:
    """
    Produce one offspring from two parents.
    Returns an unevaluated Individual (fitness = -1).
    """
    ops = []

    # -- crossover --
    if rng.random() < config.CROSSOVER_RATE:
        prompt = llm.crossover(parent_a.prompt, parent_b.prompt)
        ops.append("crossover")
    else:
        prompt = rng.choice([parent_a.prompt, parent_b.prompt])
        ops.append("copy")

    # -- mutations (independent -- all three can fire) --
    if rng.random() < config.MUT_INJECT_PROB:
        prompt = llm.mutate_inject(prompt)
        ops.append("inject")
    if rng.random() < config.MUT_DELETE_PROB:
        prompt = llm.mutate_delete(prompt)
        ops.append("delete")
    if rng.random() < config.MUT_REPHRASE_PROB:
        prompt = llm.mutate_rephrase(prompt)
        ops.append("rephrase")

    return Individual(
        prompt=prompt,
        parent_a=parent_a.uid,
        parent_b=parent_b.uid,
        operators="+".join(ops),
    )


# --------------------------
# Main GA loop
# --------------------------

def run(
    seed_prompts: list[str],
    problems: list[dict],
    on_generation=None,
```

```
) -> tuple[Individual, list[dict]]:
    """
    Run the full genetic algorithm.
    Returns: (best_individual, history)
    """
    rng = random.Random(config.RANDOM_SEED)
    history: list[dict] = []

    # -- initialise population --
    population = [Individual(prompt=p) for p in seed_prompts]
    print(f"[gen 0] evaluating {len(population)} seed individuals ..."
        )
    for ind in population:
        ind.fitness = compute_fitness(ind, problems)
        print(f"  {ind.uid}  fitness={ind.fitness:.4f}")

    for gen in range(1, config.GENERATIONS + 1):
        # -- record stats for PREVIOUS generation --
        population.sort(key=lambda x: x.fitness, reverse=True)
        fits = [x.fitness for x in population]
        stats = {
            "generation": gen - 1,
            "best": max(fits),
            "mean": sum(fits) / len(fits),
            "worst": min(fits),
            "best_uid": population[0].uid,
            "best_prompt": population[0].prompt,
        }
        history.append(stats)
        print(
            f"[gen {gen-1}] best={stats['best']:.4f}  "
            f"mean={stats['mean']:.4f}  worst={stats['worst']:.4f}"
        )

        if on_generation:
            on_generation(gen - 1, population, stats)

        # -- elitism: carry over top ELITISM_COUNT --
        new_pop = population[: config.ELITISM_COUNT]

        # -- breed offspring to fill remaining slots --
        while len(new_pop) < config.POPULATION_SIZE:
            p1 = tournament_select(population, rng)
            p2 = tournament_select(population, rng)
            child = breed(p1, p2, rng)
            new_pop.append(child)

        # -- evaluate only the new offspring --
        print(f"[gen {gen}] evaluating {config.POPULATION_SIZE -
            config.ELITISM_COUNT} offspring ...")
        for ind in new_pop[config.ELITISM_COUNT :]:
            ind.fitness = compute_fitness(ind, problems)
            print(f"  {ind.uid}  fitness={ind.fitness:.4f}  ops={ind.
                operators}")

        population = new_pop

```

```
    # -- final stats --
    population.sort(key=lambda x: x.fitness, reverse=True)
    fits = [x.fitness for x in population]
    history.append({
        "generation": config.GENERATIONS,
        "best": max(fits),
        "mean": sum(fits) / len(fits),
        "worst": min(fits),
        "best_uid": population[0].uid,
        "best_prompt": population[0].prompt,
    })
    print(
        f"[gen {config.GENERATIONS}] FINAL  best={max(fits):.4f}  "
        f"mean={sum(fits)/len(fits):.4f}"
    )

    return population[0], history
```

9.4 Component-by-Component Analysis

9.4.1 The Individual Dataclass

@dataclass class Individual

Defined: ga.py, line 35
Created by: list comprehension in `run()` (seeds); `breed()` (offspring)
Fields:
`prompt: str` — the system prompt (the chromosome)
`fitness: float = -1.0` — score in [0.0, 1.0]; -1.0 = not yet evaluated
`uid: str` — 8-char random hex ID (e.g. `"a3f7c2b1"`)
`parent_a: str = ""` — uid of first parent; empty string for seeds
`parent_b: str = ""` — uid of second parent; empty string for seeds
`operators: str = ""` — comma-separated operators (e.g. `"crossover+inject"`)

```
# Seed individual (created in run()):
Individual(
    prompt="Think like a competitive programmer...",
    fitness=-1.0,        # sentinel: not yet evaluated
    uid="a3f7c2b1",      # uuid.uuid4().hex[:8]
    parent_a="",         # no parents (seed)
    parent_b="",
    operators=""
)

# After fitness evaluation:
Individual(
    prompt="Think like a competitive programmer...",
    fitness=0.7312,      # set by compute_fitness()
    uid="a3f7c2b1",
    parent_a="",
    parent_b="",
    operators=""
)

# Offspring individual (created by breed()):
```

```
Individual(
    prompt="Approach as a competitive coder. Check edge cases...",
    fitness=-1.0,        # still unevaluated; set by compute_fitness() next
    uid="d7e1f4a2",
    parent_a="a3f7c2b1",    # uid of parent 1
    parent_b="c9b2d3f0",    # uid of parent 2
    operators="crossover+inject"
)
```

9.4.2 compute_fitness(individual, problems)

compute_fitness(individual: Individual, problems: list[dict]) -> float

Defined: ga.py, line 50
Called by: run() — for all 15 seeds at gen 0 [ga.py:170]; for 13 new offspring per generation [ga.py:206]
Calls: llm.generate_code(), evaluator.extract_code(), evaluator.run_tests()

Parameters:
individual — the Individual whose prompt is being evaluated
problems — list of 100 problem dicts (the fixed evolution pool)

Returns: float in [0.0, 1.0] — mean partial-credit across 100 problems

```
compute_fitness(individual, problems)
  |
  +-- [for each of 100 problems]:
  |
  |   1. response = llm.generate_code(            [llm.py:77]
  |                     individual.prompt,
  |                     prob["question"],
  |                     prob["fn_name"])
  |      |
  |      +-- _call(phase="fitness",               [llm.py:29]
  |                model="claude-haiku-4-5-20251001",
  |                temperature=0.0,
  |                max_tokens=1024, ...)
  |         Returns: raw LLM text string
  |
  |   2. code = evaluator.extract_code(response)  [evaluator.py:24]
  |      Returns: Python code string
  |
  |   3. score = evaluator.run_tests(             [evaluator.py:84]
  |                  code, prob["fn_name"],
  |                  prob["inputs"], prob["outputs"],
  |                  timeout=5)
  |      Returns: float in [0.0, 1.0]
  |
  |   4. total += score
  |
  +-- return total / 100   (mean across all problems)
```

compute_fitness is expensive

One call to compute_fitness makes **100 API calls** to Claude Haiku. Over a full run (15 seeds + 20 gens x 13 offspring = 275 evaluations): $275 \times 100 = 27{,}500$ Haiku API calls. This is why the fitness pool is fixed at 100 problems (not 1,000) and why Haiku (not Sonnet) is

the target model.

9.4.3 tournament_select(population, rng)

tournament_select(population: list[Individual], rng: random.Random) -> Individual

Defined: ga.py, line 84
Called by: run() — twice per offspring [ga.py:199–200]
Calls: rng.sample(), max(..., key=...)

Returns: the highest-fitness Individual among 3 randomly drawn contestants

```
# population (sorted by fitness, example values):
# [ind_A: 0.82, ind_B: 0.78, ind_C: 0.74, ind_D: 0.71, ...]

# rng.sample(population, 3) draws 3 WITHOUT replacement:
contestants = [ind_C: 0.74, ind_B: 0.78, ind_D: 0.71]

# max(..., key=lambda ind: ind.fitness) returns the best:
winner = ind_B  # fitness 0.78

# Called TWICE per offspring:
p1 = tournament_select(population, rng)  # e.g., ind_B (0.78)
p2 = tournament_select(population, rng)  # e.g., ind_A (0.82)
# p1 and p2 can be the same individual if drawn twice; this is rare
# and produces a clone (crossover of identical parents = copy).
```

Tournament selection: exploration vs. exploitation

If we always selected the globally best individual as a parent, all offspring would be children of the top 1-2 prompts — the population would converge too quickly (premature convergence) and lose diversity. Tournament selection gives every individual a chance proportional to its fitness: the best usually wins, but weaker individuals can win if they happen to be drawn when even weaker contestants are in their tournament.

9.4.4 breed(parent_a, parent_b, rng)

breed(parent_a: Individual, parent_b: Individual, rng: random.Random) -> Individual

Defined: ga.py, line 94
Called by: run() [ga.py:201]
Calls: llm.crossover(), llm.mutate_inject(), llm.mutate_delete(), llm.mutate_rephrase(), rng.random(), rng.choice()

Returns: unevaluated Individual (fitness=-1.0)

Complete decision tree for one offspring:

```
breed(parent_a, parent_b, rng):
  |
  +-- Roll rng.random() [call 1]
  |     < 0.80? --> llm.crossover(parent_a.prompt, parent_b.prompt)
  |                 ops = ["crossover"]
  |                 [1 API call to Sonnet]
```

```
|      >= 0.80? --> rng.choice([parent_a.prompt, parent_b.prompt])
|                   ops = ["copy"]
|                   [0 API calls]
|
+-- Roll rng.random() [call 2] -- INDEPENDENT
|      < 0.25? --> prompt = llm.mutate_inject(prompt)
|                  ops.append("inject")
|                  [1 API call to Sonnet]
|
+-- Roll rng.random() [call 3] -- INDEPENDENT
|      < 0.15? --> prompt = llm.mutate_delete(prompt)
|                  ops.append("delete")
|                  [0 or 1 API calls, depending on sentence count]
|
+-- Roll rng.random() [call 4] -- INDEPENDENT
       < 0.10? --> prompt = llm.mutate_rephrase(prompt)
                   ops.append("rephrase")
                   [1 API call to Sonnet]

--> return Individual(
         prompt=prompt,
         parent_a=parent_a.uid,   # genealogy tracking
         parent_b=parent_b.uid,
         operators="+".join(ops)  # e.g. "crossover+inject" or "copy+rephrase"
     )
# fitness is NOT set here; run() calls compute_fitness() separately
```

9.4.5 run(seed_prompts, problems, on_generation=None)

run(seed_prompts, problems, on_generation=None) -> tuple[Individual, list[dict]]

Defined: ga.py, line 146
Called by: main.main() [main.py:120]
Calls: compute_fitness(), tournament_select(), breed(), on_generation callback (if provided)

Parameters:
seed_prompts — list of 15 starting prompt strings
problems — 100-problem evolution pool
on_generation — optional callback invoked after each generation

Returns: (best_Individual, history: list[dict])

The main loop structure, annotated:

```
run(seed_prompts, problems, on_generation):

  [SETUP]
  rng = random.Random(42)  # seeded for reproducibility
  history = []

  [GEN 0: Evaluate seeds]
  population = [Individual(prompt=p) for p in seed_prompts]  # 15 individuals
  for ind in population:                                      # 15 iterations
      ind.fitness = compute_fitness(ind, problems)           # 100 API calls
         each

  [GENS 1..20: The evolutionary loop]
  for gen in range(1, 21):                                    # 20 iterations
```

```
    [RECORD STATS FOR PREVIOUS GENERATION]
    population.sort(by fitness, descending)
    stats = {generation: gen-1, best: ..., mean: ..., worst: ..., ...}
    history.append(stats)
    if on_generation: on_generation(gen-1, population, stats)
    # => saves generation_(gen-1).json checkpoint

    [ELITISM: copy top 2 unchanged]
    new_pop = population[:2]   # best 2 individuals copied verbatim

    [BREED: fill remaining 13 slots]
    while len(new_pop) < 15:
        p1 = tournament_select(population, rng)  # pick 2 parents
        p2 = tournament_select(population, rng)
        child = breed(p1, p2, rng)               # crossover + mutations
        new_pop.append(child)                     # child.fitness = -1.0

    [EVALUATE: only the 13 new offspring]
    for ind in new_pop[2:]:                       # skip the 2 elites
        ind.fitness = compute_fitness(ind, problems)  # 100 API calls each

    population = new_pop   # elites (re-used) + 13 new evaluated offspring

[FINAL STATS]
population.sort(by fitness, descending)
history.append({generation: 20, best: ..., mean: ..., ...})

return (population[0], history)   # best individual + all 21 stat records
```

History record structure

The `history` list contains 21 entries (generations 0 through 20). Each entry is:

```
{
    "generation": 0,          # or 1, 2, ..., 20
    "best": 0.7312,           # highest fitness in population at this gen
    "mean": 0.6841,           # average fitness
    "worst": 0.5920,          # lowest fitness
    "best_uid": "a3f7c2b1",   # uid of the current best individual
    "best_prompt": "Think like a competitive programmer..."
}
```

Generations 0–19 are recorded at the *start* of the following generation's loop (i.e. `stats["generation"]` = `gen - 1`). Generation 20's stats are recorded after the loop ends.

Generational replacement

Each generation, the *entire* population is replaced (except the 2 elites). This is called **full generational replacement**. The alternative is *steady-state*, where only 1–2 individuals are replaced per step. Full replacement creates strong selection pressure: any individual not in the top 2 must produce offspring that beat their parents to survive. This speeds up convergence at the cost of some diversity. Elitism (top 2 guaranteed) prevents this from becoming too aggressive.

Chapter 10

Entry Point: `main.py`

10.1 Purpose

This file orchestrates the entire run. It is the only file you call directly (`python main.py`). Its job is to wire together all other modules in the correct order, checkpoint progress after each generation, run the final holdout evaluation, and write the cost report.

10.2 File at a Glance

Item	Type	Description
`save_checkpoint()`	function	Write full population state to a per-generation JSON file
`evaluate_on_holdout()`	function	Run best prompt on 200 held-out problems
`main()`	function	The 7-step orchestration pipeline

Table 10.1: `main.py`: public items.

10.3 Full Source Code

Listing 10.1: `main.py` — complete source.

```python
"""
main.py -- Entry point.  Load data -> run GA -> save results -> save cost report.

Usage:
    export ANTHROPIC_API_KEY="sk-ant-..."
    python main.py

Results are saved to Code/results/run_<timestamp>/
Cost reports are saved to Code/Costs/cost_<timestamp>.json  (new file each run)
"""

import json
import sys
import time
from datetime import datetime
from pathlib import Path

import config
import dataset
import evaluator
```

```python
import ga
import llm
from cost_tracker import tracker
from seeds import SEED_PROMPTS


def save_checkpoint(run_dir: Path, gen: int, population, stats):
    """Save population state after each generation for crash recovery.
    """
    gen_file = run_dir / f"generation_{gen:02d}.json"
    data = {
        "generation": gen,
        "stats": stats,
        "population": [
            {
                "uid": ind.uid,
                "prompt": ind.prompt,
                "fitness": ind.fitness,
                "parent_a": ind.parent_a,
                "parent_b": ind.parent_b,
                "operators": ind.operators,
            }
            for ind in population
        ],
    }
    gen_file.write_text(json.dumps(data, indent=2))


def evaluate_on_holdout(best_prompt: str, holdout: list[dict]) ->
    float:
    """Evaluate the best evolved prompt on the held-out test set."""
    total = 0.0
    for i, prob in enumerate(holdout):
        response = llm.generate_code(best_prompt, prob["question"],
            prob["fn_name"])
        code = evaluator.extract_code(response)
        score = evaluator.run_tests(
            code, prob["fn_name"], prob["inputs"], prob["outputs"],
                config.EXEC_TIMEOUT,
        )
        total += score
        if (i + 1) % 50 == 0:
            print(f"  holdout progress: {i+1}/{len(holdout)}")
    return total / len(holdout)


def main():
    # -- pre-flight checks --
    if not config.ANTHROPIC_API_KEY:
        print("ERROR: set ANTHROPIC_API_KEY environment variable.")
        sys.exit(1)

    if not config.APPS_TRAIN.exists():
        print(f"ERROR: APPS train directory not found: {config.
            APPS_TRAIN}")
        sys.exit(1)

```

```python
    # -- load and split dataset --
    print("Loading APPS Codewars introductory problems ...")
    all_problems = dataset.load_problems(config.APPS_TRAIN)
    print(f"  Loaded {len(all_problems)} problems")

    if len(all_problems) < config.EVOLUTION_SIZE + config.HOLDOUT_SIZE:
        print("ERROR: not enough problems after filtering.")
        sys.exit(1)

    evolution_pool, holdout, rest = dataset.split(
        all_problems, config.EVOLUTION_SIZE, config.HOLDOUT_SIZE, config.RANDOM_SEED,
    )
    print(f"  Evolution pool : {len(evolution_pool)} problems")
    print(f"  Held-out test  : {len(holdout)} problems")
    print(f"  Remaining      : {len(rest)} problems")

    # -- create run directory --
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    run_dir = config.RESULTS_DIR / f"run_{timestamp}"
    run_dir.mkdir(parents=True, exist_ok=True)

    # save config snapshot
    (run_dir / "config.json").write_text(json.dumps({
        "population_size":  config.POPULATION_SIZE,
        "generations":      config.GENERATIONS,
        "crossover_rate":   config.CROSSOVER_RATE,
        "tournament_size":  config.TOURNAMENT_SIZE,
        "elitism_count":    config.ELITISM_COUNT,
        "mut_inject":       config.MUT_INJECT_PROB,
        "mut_delete":       config.MUT_DELETE_PROB,
        "mut_rephrase":     config.MUT_REPHRASE_PROB,
        "evolution_size":   config.EVOLUTION_SIZE,
        "holdout_size":     config.HOLDOUT_SIZE,
        "target_model":     config.TARGET_MODEL,
        "optimizer_model":  config.OPTIMIZER_MODEL,
        "random_seed":      config.RANDOM_SEED,
    }, indent=2))

    # save evolution problem IDs for reproducibility
    (run_dir / "evolution_ids.json").write_text(
        json.dumps([p["id"] for p in evolution_pool])
    )

    print(f"\nResults will be saved to: {run_dir}\n")

    # -- run GA --
    start = time.time()
    best, history = ga.run(
        seed_prompts=SEED_PROMPTS,
        problems=evolution_pool,
        on_generation=lambda gen, pop, stats: save_checkpoint(run_dir, gen, pop, stats),
    )
    elapsed = time.time() - start

```

```
    # -- save results --
    (run_dir / "history.json").write_text(json.dumps(history, indent=2))
    (run_dir / "best_prompt.txt").write_text(best.prompt)

    print(f"\n{'='*60}")
    print(f"GA complete in {elapsed/60:.1f} minutes")
    print(f"Best fitness (evolution pool): {best.fitness:.4f}")
    print(f"Best prompt:\n{best.prompt}")
    print(f"{'='*60}\n")

    # -- evaluate best prompt on held-out set --
    print(f"Evaluating best prompt on {len(holdout)} held-out problems...")
    holdout_fitness = evaluate_on_holdout(best.prompt, holdout)
    print(f"Held-out fitness: {holdout_fitness:.4f}")

    # -- save final summary --
    summary = {
        "elapsed_minutes": round(elapsed / 60, 1),
        "best_prompt": best.prompt,
        "best_fitness_evolution": best.fitness,
        "best_fitness_holdout": holdout_fitness,
        "best_uid": best.uid,
        "total_generations": config.GENERATIONS,
    }
    (run_dir / "summary.json").write_text(json.dumps(summary, indent=2))

    # -- save cost report to Code/Costs/ --
    cost_path = tracker.flush(extra={
        "run_dir": str(run_dir),
        "best_fitness_evolution": best.fitness,
        "best_fitness_holdout": holdout_fitness,
        "generations": config.GENERATIONS,
        "population_size": config.POPULATION_SIZE,
        "evolution_size": config.EVOLUTION_SIZE,
    })
    print(f"\nCost report saved to: {cost_path}")
    print(f"Total cost this run:  ${tracker.total():.2f}")
    print(f"\nAll results saved to: {run_dir}")


if __name__ == "__main__":
    main()
```

10.4 Function-by-Function Analysis

10.4.1 save_checkpoint(run_dir, gen, population, stats)

```
save_checkpoint(run_dir: Path, gen: int, population: list[Individual],
stats: dict)
```

Defined: main.py, line 27
Called by: the lambda in ga.run()'s on_generation callback [main.py:123], which is invoked by ga.run() as on_generation(gen-1, population, stats) at the *start* of each breeding step — meaning it saves the state *before* the new offspring are created.

Calls: json.dumps(), Path.write_text()
Output: run_dir/generation_NN.json (e.g., generation_04.json)

What is written:

```
# generation_04.json (after generation 4 is complete):
{
    "generation": 4,
    "stats": {
        "generation": 4,
        "best": 0.7891,
        "mean": 0.7234,
        "worst": 0.6103,
        "best_uid": "d7e1f4a2",
        "best_prompt": "Approach as a competitive coder..."
    },
    "population": [
        {
            "uid": "d7e1f4a2",
            "prompt": "Approach as a competitive coder...",
            "fitness": 0.7891,
            "parent_a": "a3f7c2b1",
            "parent_b": "c9b2d3f0",
            "operators": "crossover+inject"
        },
        ...  # all 15 individuals
    ]
}
```

Why checkpointing matters: A full GA run takes several hours (27,500+ API calls). If the process crashes at generation 15, without checkpoints the entire run must restart from generation 0. With checkpoints, you can inspect which prompts were alive at each generation and understand the evolutionary trajectory.

10.4.2 evaluate_on_holdout(best_prompt, holdout)

```
evaluate_on_holdout(best_prompt: str, holdout: list[dict]) -> float
```

Defined: main.py, line 48
Called by: main() [main.py:139]
Calls: llm.generate_code(), evaluator.extract_code(), evaluator.run_tests()

Parameters:
best_prompt — the winning chromosome from ga.run()
holdout — the 200 never-seen-during-evolution problems

Returns: `float` — mean partial-credit across 200 holdout problems

This function is structurally identical to `ga.compute_fitness()` but runs on the 200-problem holdout set instead of the 100-problem evolution set. The only added detail is a progress print every 50 problems (line 59), since 200 sequential API calls take a few minutes.

```
evaluate_on_holdout(best_prompt, holdout):
  |
  +-- [for i, prob in enumerate(holdout)]:  # 200 iterations
  |
  |   response = llm.generate_code(best_prompt, prob["question"], prob["fn_name
     "])
  |              [llm.py:77 => 1 Haiku API call per problem]
  |
  |   code = evaluator.extract_code(response)   [evaluator.py:24]
  |
  |   score = evaluator.run_tests(...)           [evaluator.py:84]
  |
  |   total += score
  |
  |   if (i+1) % 50 == 0: print progress       # every 50 problems
  |
  +-- return total / 200
```

10.4.3 `main()` — The 7-Step Pipeline

`main() -> None`

Defined: `main.py`, line 63
Called by: `if __name__ == "__main__": main()` [main.py:167]
Calls: everything in the project

Step 1 — Pre-flight checks (lines 65–71):

```
if not config.ANTHROPIC_API_KEY:
    sys.exit(1)         # fail fast if no API key
if not config.APPS_TRAIN.exists():
    sys.exit(1)         # fail fast if dataset not found
# These checks happen BEFORE any expensive operations.
```

Step 2 — Load and split dataset (lines 74–87):

```
all_problems = dataset.load_problems(config.APPS_TRAIN)    # ~2346 problems
evolution_pool, holdout, rest = dataset.split(
    all_problems, 100, 200, seed=42)
# evolution_pool: the 100 problems the GA will train on
# holdout:        200 unseen problems for final test
```

Step 3 — Create run directory (lines 90–114):

```
run_dir = config.RESULTS_DIR / f"run_{timestamp}"
# e.g., Code/results/run_20260417_142035/
run_dir.mkdir(parents=True, exist_ok=True)
# Saves immediately:
#   config.json        -- exact hyperparameters used
#   evolution_ids.json -- which 100 problem IDs were used
```

Step 4 — Run GA (lines 119–125):

```
best, history = ga.run(
```

```
    seed_prompts=SEED_PROMPTS,       # the 15 starting chromosomes
    problems=evolution_pool,          # fixed 100-problem pool
    on_generation=lambda gen, pop, stats:
        save_checkpoint(run_dir, gen, pop, stats)
)
# ga.run() takes hours; save_checkpoint() runs after each generation
# so crash recovery loses at most one generation of work.
```

Step 5 — Save results (lines 128–135):

```
(run_dir / "history.json").write_text(json.dumps(history, indent=2))
# history: list of 21 dicts {generation, best, mean, worst, best_uid,
    best_prompt}

(run_dir / "best_prompt.txt").write_text(best.prompt)
# The evolved prompt -- the main artifact of the experiment.
```

Step 6 — Holdout evaluation (lines 138–140):

```
holdout_fitness = evaluate_on_holdout(best.prompt, holdout)
# 200 more API calls to Haiku. Measures generalisation.
# If evolution fitness >> holdout fitness: the prompt overfitted to the 100
    problems.
```

Step 7 — Save cost report (lines 154–163):

```
cost_path = tracker.flush(extra={
    "run_dir": str(run_dir),
    "best_fitness_evolution": best.fitness,
    "best_fitness_holdout": holdout_fitness,
    ...
})
# Writes: Code/Costs/cost_20260417_142035.json
# Contains complete token-by-token and phase-by-phase breakdown.
```

10.5 Output Files Reference

File	Contents
`results/run_*/config.json`	All hyperparameters used in this run (reproducibility)
`results/run_*/evolution_ids.json`	The exact 100 APPS problem IDs used as the fitness pool
`results/run_*/generation_NN.json`	Full population state (prompts, fitnesses, lineage) per gen
`results/run_*/history.json`	Best/mean/worst fitness per generation (21 entries)
`results/run_*/best_prompt.txt`	The winning prompt (main artifact)
`results/run_*/summary.json`	Final fitnesses (evolution + holdout), best prompt, runtime
`Costs/cost_*.json`	Full API cost breakdown by phase and model

Table 10.2: Output files produced by each run.

Chapter 11

Complete Worked Example: One Problem Through the Pipeline

This chapter traces a single coding problem through the *entire* pipeline — from disk to a fitness score — using concrete data at every step. The goal is to show exactly what data each function receives and returns.

11.1 The Problem (on disk)

```
# Problem folder: Dataset Examples/APPS/train/3715/

# metadata.json:
{
    "difficulty": "introductory",
    "url": "https://www.codewars.com/kata/572b6b2772a32e92e900010b",
    "source": "codewars"
}

# question.txt:
"Complete the function that takes an array of words.
 You must concatenate the nth letter from each word using the enumerating order
 of the word as n.
 If any word has fewer letters than the enumerating order value, use '#'.

 Example: nth_char(['yoda','best','has'])   == 'oes'
          nth_char(['a', 'bc', 'def'])       == 'abd'"

# input_output.json:
{
    "fn_name": "nth_char",
    "inputs":  [
        [["yoda", "best", "has"]],
        [["a", "bc", "def"]]
    ],
    "outputs": [
        ["oes"],
        ["abd"]
    ]
}
```

11.2 Step 1: `dataset.load_problems()` reads the problem

```
# After the filter chain passes (introductory, fn_name, codewars):
problem = {
    "id":       "3715",
    "question": "Complete the function that takes an array of words.\n"
                "You must concatenate the nth letter from each word...",
    "fn_name":  "nth_char",
```

```
    "inputs":   [
        [["yoda", "best", "has"]],    # test 1: one arg (a list of 3 words)
        [["a", "bc", "def"]]          # test 2: one arg (a list of 3 words)
    ],
    "outputs":  [
        ["oes"],                        # test 1 expected: 'oes'
        ["abd"]                         # test 2 expected: 'abd'
    ]
}
# This dict is one entry in the 2346-item list returned by load_problems().
```

11.3 Step 2: `dataset.split()` assigns this problem to the evolution pool

```
# After shuffle with seed=42, problem "3715" lands in position 47:
evolution_pool = shuffled[:100]    # positions 0..99
# Problem "3715" is at position 47 => it is in evolution_pool.
# This same 100-problem pool is used for EVERY generation.
```

11.4 Step 3: `ga.compute_fitness()` processes the problem

```
# Individual being evaluated:
individual = Individual(
    prompt="Think like a competitive programmer. Identify the algorithm "
           "pattern (math, string manipulation, recursion, sorting, hashing). "
           "Write the most direct, correct Python solution. Output only the "
           "function definition.",
    fitness=-1.0,  # not yet evaluated
    uid="a3f7c2b1"
)

# The for loop reaches problem "3715":
prob = evolution_pool[47]    # the nth_char problem
```

11.5 Step 4: `llm.generate_code()` sends the API request

```
# Arguments passed to generate_code():
system_prompt = individual.prompt    # the competitive programmer prompt
question      = prob["question"]     # "Complete the function that takes..."
fn_name       = "nth_char"

# user message assembled inside generate_code():
user_msg = (
    "Complete the function that takes an array of words.\n"
    "You must concatenate the nth letter from each word using the "
    "enumerating order of the word as n.\n"
    "If any word has fewer letters than the enumerating order value, use '#'.\n\
       n"
    "Example: nth_char(['yoda','best','has'])    == 'oes'\n"
    "         nth_char(['a', 'bc', 'def'])        == 'abd'\n\n"
    "Write a Python function named `nth_char`."
)
```

```
# This is passed to _call() which sends to Claude Haiku 4.5:
# model=claude-haiku-4-5-20251001, temperature=0.0, max_tokens=1024
```

11.6 Step 5: `llm._call()` makes the API request and logs cost

```
# Anthropic API call (simplified):
resp = client.messages.create(
    model="claude-haiku-4-5-20251001",
    max_tokens=1024,
    temperature=0.0,          # deterministic: always same output for same input
    system="Think like a competitive programmer...",
    messages=[{"role": "user", "content": user_msg}]
)

# resp.usage:
#   input_tokens  = 287   (system prompt ~80 tokens + user message ~207 tokens)
#   output_tokens =  52   (the generated function body)
#
# resp.content[0].text (the LLM's raw response):
raw_response = """```python
def nth_char(words):
    return ''.join(
        word[i] if i < len(word) else '#'
        for i, word in enumerate(words)
    )
```"""

Cost logged:
tracker.record(
    phase="fitness",
    model="claude-haiku-4-5-20251001",
    input_tokens=287,
    output_tokens=52
)
input_cost = 287 * 1.00 / 1_000_000 = $0.000287
output_cost = 52 * 5.00 / 1_000_000 = $0.000260
total_cost = $0.000547 for this one call
```

## 11.7 Step 6: `evaluator.extract_code()` strips the markdown

```
response = """```python
def nth_char(words):
    return ''.join(
        word[i] if i < len(word) else '#'
        for i, word in enumerate(words)
)
```"""

# Strategy 1 fires: regex r"```python\s*\n([\s\S]*?)```"
# Captures group 1 = everything between the fences:
code = "def nth_char(words):\n    return ''.join(\n        word[i] if i < len(
    word) else '#'\n        for i, word in enumerate(words)\n    )"
```

11.8 Step 7: `evaluator.run_tests()` executes and scores

```
# has_function check:
re.search(r"\bdef\s+nth_char\s*\(", code)  => MATCH => True
# Proceed.

# _build_script produces this script:
"""
import sys
sys.set_int_max_str_digits(1_000_000)

def nth_char(words):
    return ''.join(
        word[i] if i < len(word) else '#'
        for i, word in enumerate(words)
    )

def __run__(inp, expected):
    try:
        result = nth_char(*inp)
        sys.stdout.write("__R__:" + ("1" if [result] == expected else "0") + "\n
            ")
        sys.stdout.flush()
    except Exception:
        sys.stdout.write("__R__:0\n")
        sys.stdout.flush()

__run__([["yoda", "best", "has"]], ["oes"])
__run__([["a", "bc", "def"]], ["abd"])
"""

# subprocess.run([python3, "-c", script], timeout=5) executes this.

# Execution trace:
# Test 1: nth_char(*[["yoda","best","has"]]) = nth_char(["yoda","best","has"])
#   i=0, word="yoda" => word[0] = 'y'  -- wait, 'y' not 'o'?
#   Hmm, let me recheck: nth_char(['yoda','best','has']) == 'oes'
#   i=0, word="yoda" => word[0] = 'y'  ... but expected 'o'
#   Wait -- "enumerating order of the word" means n=index of the word, not char.
#   i=0 => word[0] from "yoda" = 'y'
#   i=1 => word[1] from "best" = 'e'
#   i=2 => word[2] from "has"  = 's'
#   Result: "yes" -- but expected "oes"
#   Hmm, so the expected output is 'oes':
#     nth_char(['yoda','best','has']):
#       i=0, word='yoda' => word[0]='y'? Or word[1]='o'?
#   Actually re-reading: "nth letter from each word using the enumerating order"
#   This means: word 0 -> take letter at index 1 (0+1)?
#   No wait: 'yoda'[0]='y', 'best'[1]='e', 'has'[2]='s' => 'yes' != 'oes'
#   Or: 'yoda'[1]='o', 'best'[2]='e'... wait 'best'[2]='s' not 'e'
#   Or: 'yoda'[0+1?]...
#   Actually checking: 'oes': o from yoda (index 1?), e from best (index 1?), s
    from has (index 1?)
#   No: 'yoda'[1]='o', 'best'[1]='e', 'has'[1]='a' => 'oea' != 'oes'
#   Let me try: y[1]='o', b[0]='b', h[2]='s'...
#   Actually the correct interpretation may be that n starts from 1 (1-indexed):
#   i=1, word="yoda" => word[1]='o'; i=2, word="best" => word[2]='e'; i=3(>len("
    has")=3) => word[2]='s'
#   "has"[2]='s' -- yes! n=3, index=2
#   So: 1-indexed: yoda[1]='o', best[2]='e', has[3]='s' => 'oes' - correct!
```

```
#   The code above uses 0-based indexing (i starts from 0), which would give '
    yes', not 'oes'.
#   A correct implementation should use 1-based indexing:
#     for i, word in enumerate(words, 1):
#         word[i-1] if i <= len(word) else '#'
#   OR equivalently enumerate starting from 1.

# So the code generated by the LLM is WRONG for this problem.
# Let's say it produces INCORRECT results:
# Test 1: result = "yes", expected = "oes" => [result]=="yes" != ["oes"] =>
    __R__:0
# Test 2: nth_char(["a","bc","def"]):
#   0-based: a[0]='a', bc[1]='c', def[2]='f' => 'acf' vs expected 'abd'
#   __R__:0

# subprocess stdout: "__R__:0\n__R__:0\n"
# results = [0, 0]
# score = sum([0,0]) / len(inputs) = 0 / 2 = 0.0

# This is a FITNESS SIGNAL: the competitive programmer prompt scored 0.0
# on THIS problem because Haiku (at temp=0) wrote 0-indexed code.
# The GA will explore other prompts that might lead to 1-indexed solutions.
```

The worked example above shows a failure case — intentionally

This is pedagogically useful: it shows the entire pipeline completing correctly even when the *generated code is wrong*. The pipeline does not crash; it returns 0.0 for this problem and continues to the next 99 problems. The prompt's overall fitness is the *mean* across all 100 — so 0.0 on this one problem costs $1/100 = 0.01$ of the total fitness.

11.9 Step 8: The score accumulates in `compute_fitness()`

```
# After processing all 100 problems in evolution_pool:
# (hypothetical scores for the competitive programmer prompt)
scores = [0.0, 1.0, 0.5, 1.0, 0.75, 0.0, 1.0, ...]    # 100 values
total  = sum(scores)    # e.g., 73.12
fitness = total / 100   # = 0.7312

# This is stored:
individual.fitness = 0.7312

# The individual is now evaluated; ga.run() proceeds to the next seed.
```

11.10 Step 9: The result is recorded in the cost tracker

```
# After ALL 100 problems for this one individual:
# 100 entries in tracker.calls with phase="fitness"
# Typical token counts per call:
#   input: ~287 tokens (system ~80 + problem ~207)
#   output: ~52 tokens (function body)
# Total for this individual:
#   input_tokens  = 100 * 287 = 28,700
#   output_tokens = 100 *  52 =  5,200
#   input_cost    = 28700 * 1.00 / 1e6 = $0.0287
#   output_cost   =  5200 * 5.00 / 1e6 = $0.0260
#   individual_cost = ~$0.0547
```

Glossary

Chromosome A candidate solution in a genetic algorithm. In this project, each chromosome is a natural-language system prompt (a paragraph of English instructions). Stored as an `Individual` object in `ga.py`.

Crossover A genetic operator that combines two parent chromosomes into one offspring. Here, the Optimizer LLM (Sonnet 4.6) *semantically blends* the two parents rather than doing a mechanical string cut. Fires with probability 0.8 per offspring.

Deterministic (temperature = 0)
When a Claude model is called with `temperature=0.0`, the same input always produces the same output. This is why the Target model uses temperature 0: fitness scores are repeatable and comparable across generations.

Elitism Preserving the top k individuals unchanged into the next generation, preventing loss of the best prompt discovered so far. `ELITISM_COUNT = 2` in this project.

Evolution pool
The fixed set of 100 Codewars introductory coding problems used to evaluate fitness during the GA. Chosen by deterministic shuffle (seed 42) of all qualified APPS problems.

Exponential backoff
A retry strategy where the wait time doubles with each failed attempt. Used in `llm._call()` for rate-limit errors: 5s, 10s, 20s.

Fitness A numeric score in [0.0, 1.0] measuring how good a chromosome is. Fitness = mean partial-credit score across 100 evolution problems. Computed by `ga.compute_fitness()`.

`fn_name` The Python function name that the LLM must write. Stored in each APPS problem's `input_output.json`. Used to construct the user message and to verify the generated code (`evaluator.has_function()`).

Generation One cycle of the GA: record stats → carry elites → select parents → breed offspring → evaluate offspring → replace population.

Generational replacement
Replacing the entire population each generation (except elites), as opposed to steady-state (replacing 1–2 individuals at a time). Creates strong selection pressure.

Held-out set 200 problems reserved for evaluating the final evolved prompt. Never seen during evolution. Analogous to a test set in machine learning.

Individual One member of the GA population. A Python `@dataclass` with fields: `prompt`, `fitness`, `uid`, `parent_a`, `parent_b`, `operators`.

Lazy initialisation
Creating an object only when it is first needed. Used for the Anthropic client in `llm.py`: `_client` is `None` at module load; created on the first `_call()`.

LLM Large Language Model — a neural network trained on text that can generate code, merge prompts, etc. This project uses Claude Haiku 4.5 (target) and Claude Sonnet 4.6 (optimizer).

Meta-prompt A prompt sent to the Optimizer LLM instructing it to modify another prompt. The crossover and mutation functions in `llm.py` use meta-prompts to direct the Optimizer.

MTok Million tokens — the billing unit for LLM APIs. The pricing formula is: cost = (input_tokens × rate_in + output_tokens × rate_out) / 1,000,000.

Mutation A genetic operator that modifies a single chromosome. Three independent types: inject (add strategy sentence, 25%), delete (remove weakest sentence, 15%), rephrase (reword, 10%).

Partial credit Scoring a problem as (tests passed / total tests) rather than binary all-or-nothing. Provides a smooth gradient for the GA to optimise. Computed by `evaluator.run_tests()`.

pass@1 The standard code-generation evaluation metric: fraction of problems where the LLM generates correct code on its first (and only) attempt.

Population The set of all 15 chromosomes alive at a given generation. Maintained as `list[Individual]` in `ga.run()`.

Seed prompt One of the 15 hand-crafted prompts in `seeds.py` that form the generation-0 population. Each emphasises a different coding strategy.

Sentinel A unique string prefix (`__R__:`) used to tag test results in the subprocess's stdout, so they can be found among any stray `print()` output from generated code.

Singleton A class of which only one instance exists. The `tracker` object in `cost_tracker.py` is a module-level singleton; all modules that import it share the same instance.

System prompt An instruction paragraph sent *before* the user message in a Claude conversation, shaping the model's behaviour across all subsequent messages. In this project, the system prompt is the GA chromosome.

Temperature An LLM parameter controlling output randomness. 0.0 = fully deterministic; higher values introduce more variation. Target model uses 0.0; Optimizer uses 0.7.

Tournament selection A parent-selection strategy: sample k individuals randomly (without replacement) and return the fittest among them. `TOURNAMENT_SIZE = 3` in this project.

Two-model architecture Using one model (Haiku, cheap, deterministic) as the subject of optimisation and another (Sonnet, capable, creative) as the tool for genetic operators.